U O W

CSIT305 Emerging Information Technologies and their Applications

UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Software Engineering in Cloud Computing I

UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Subject Logistics

(1) Advanced Database Overview
(2) Spatial Data and Computing
(3) Graph Data and Computing
(4) Text Data and Processing

Multi-Modal Database

(1) AI4SE: artificial intelligence for software engineering
(2) Software Engineering in Cloud Computing I
(3) Software Engineering in Cloud Computing II

Software

(1) IoT and Edge Computing
(2) Digital Twin Technology
(3) Blockchains and Smart Contracts
(4) Emerging Cyber-Attacks

Networking

Outline

- Background and Preliminary
- Requirements Engineering for Cloud Application
- Cloud-Enabled Software Development
- Security Challenges in Software Engineering for the Cloud

Background - Software Development

- Software development methodologies (SDM) have evolved extensively over the past five decades. SDM applications are developed, tested, deployed, and maintained using distinct phases that structure, plan, and control the development process and the software's life cycle. These phases are referred to as the software development life cycle (SDLC), which differs based on the software development methodology (e.g., waterfall, spiral) used.

- There are various SDLCs, each with a series of processes to be followed that diverge from iterative to sequential to V-shaped. Later, non-traditional methodologies (i.e., agile or pattern-less), such as Scrum and Extreme Programming (XP), were introduced to promote practices such as test-driven development, refactoring, and pair programming among others.

- Software development methodologies (i.e., traditional and agile) are composed of five typical phases: requirements, design, implementation, testing, and maintenance. Software manufacturers generally adopt SDLCs to develop software applications depending on the size and nature of the project. Some software manufacturers employ their own software development methodology or adapt an existing one, such as the well-known Rational Unified Process (RUP), introduced by IBM in 2003

Background - Software Development

- Software applications are typically delivered either as Web-based or standalone desktop applications. Since the inception of cloud computing (CC), the software-as-a-service (SaaS) paradigm has become a standard delivery model for applications in different fields. Although there has been much debate about cloud applications, scholars from research and industry commonly accept that Web-based applications from the olden days are also considered to be cloud applications.

- However, cloud applications can be a lot more complicated than conventional Web-based applications. This is attributed to their cutting-edge and innovative concepts and technologies (e.g., multi-tenancy, load-balancing, virtualization, etc.).

- For example, some cloud applications share a physical machine with other tenants' applications. Also, cloud applications might request more resources and therefore scale up. These cases and many others have led to a new set of considerations that increase the complexity involved in cloud application manufacturing.

Background - Cloud Computing

- Cloud computing is a set of technical concepts that allow data, applications, and services which are running on several computers that are distributed somewhere in the world, but are just interconnected through some real-time communication network like the Internet.

- The National Institute of Standards and Technology (NIST) defines cloud computing as:
  - *"a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction"*

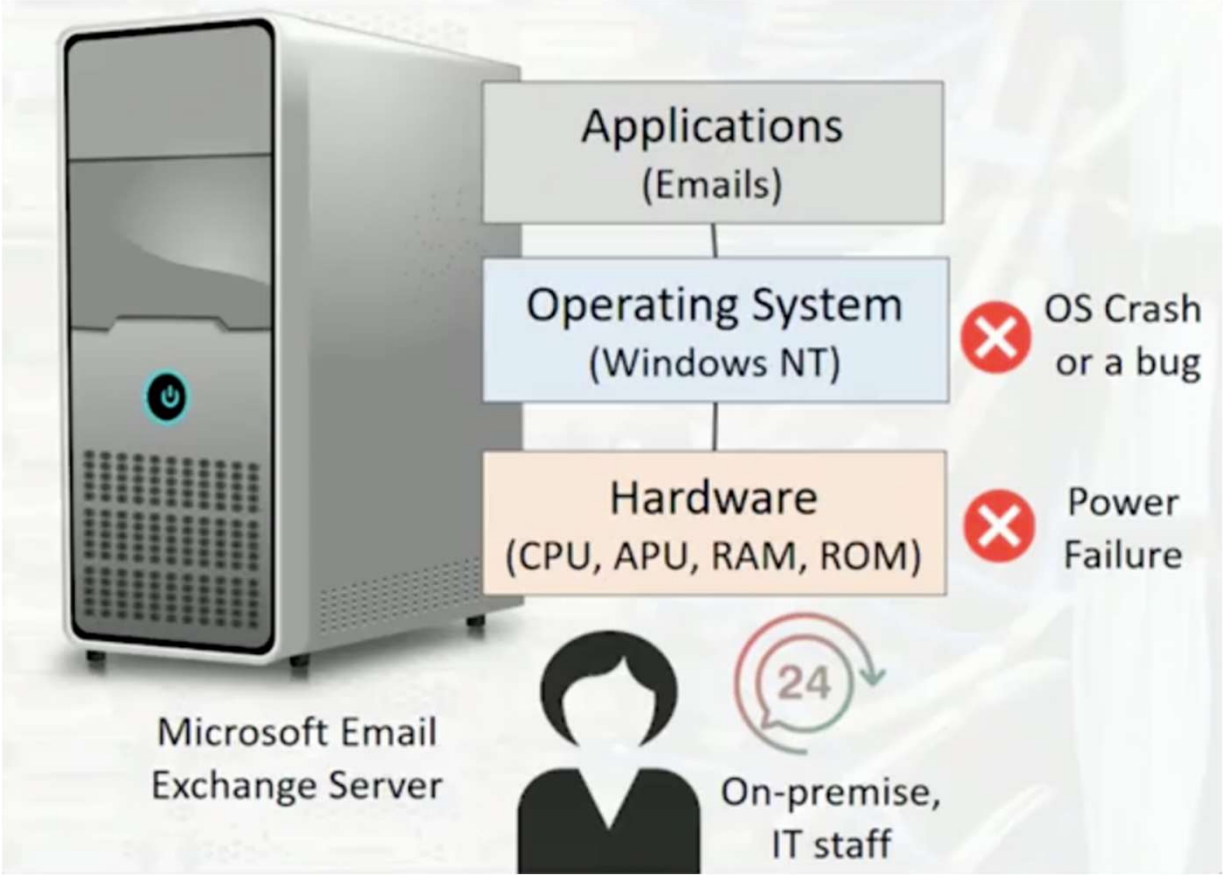
Background - Cloud Computing

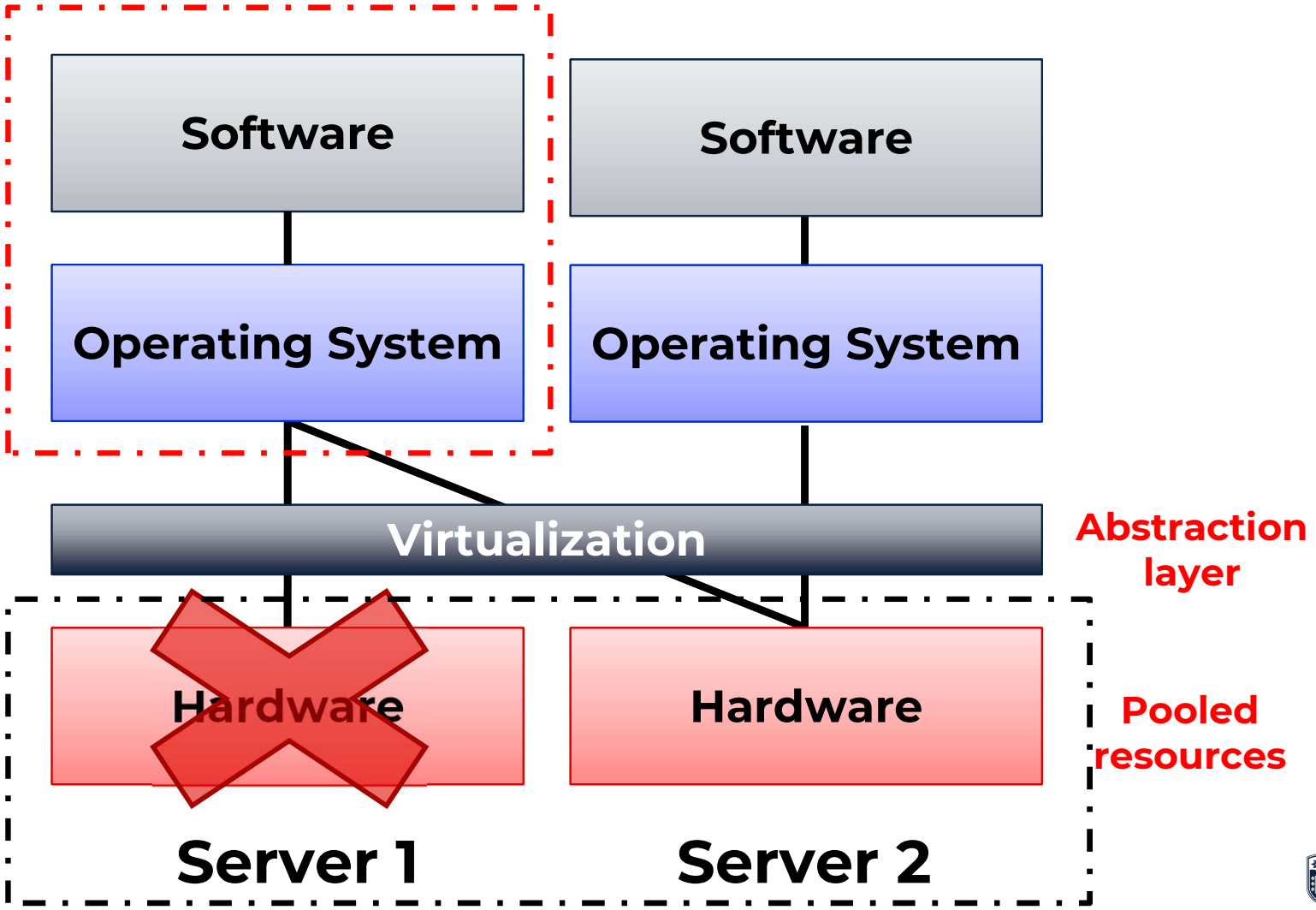
- Self-service and on-demand
  - Can sign up the service at any time, and directly start using those resources
    - E.g., Amazon web services, Microsoft Azure and Google Cloud
- Usage/metered billing
  - “Pay as you go” model: only pay the amount that you are using
    - Beneficial for small and medium-sized companies
- Broad network access
  - Can be accessed from many different types of devices from any place in the world
    - E.g., Web browsers, mobile phones, tablets

Background - Cloud Computing

- Multi-tenancy/Resource pooling
  - Shared resources are catered to many users at the same time
- Rapid elasticity/Agility
  - Provider can efficiently allocate slice(s) of unused computing resources to other tenants
    - E.g., in case of higher demand, easy to re-allocate;
    - E.g., release resources when don't need them
- Reliability
  - No need to worry about data backup

Traditional Computing Model



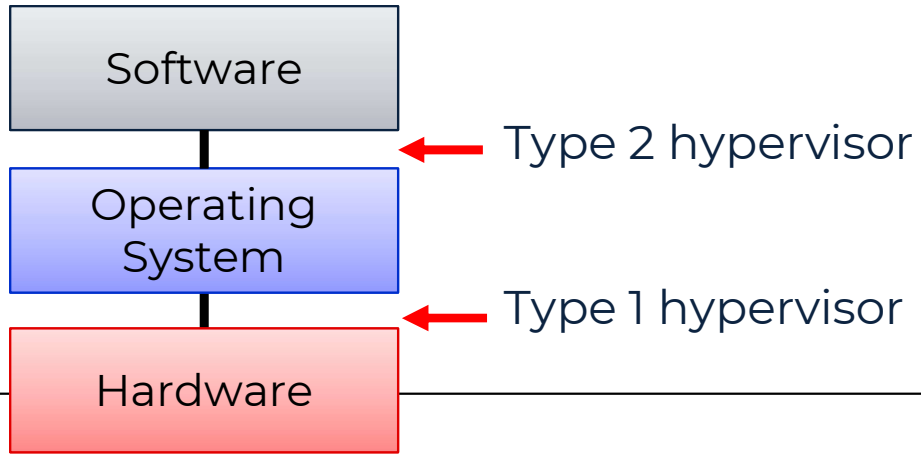

Software
Software
Operating System
Operating System
Virtualization
Abstraction layer
Hardware
Hardware
Pooled resources
Server 1
Server 2

Virtualization

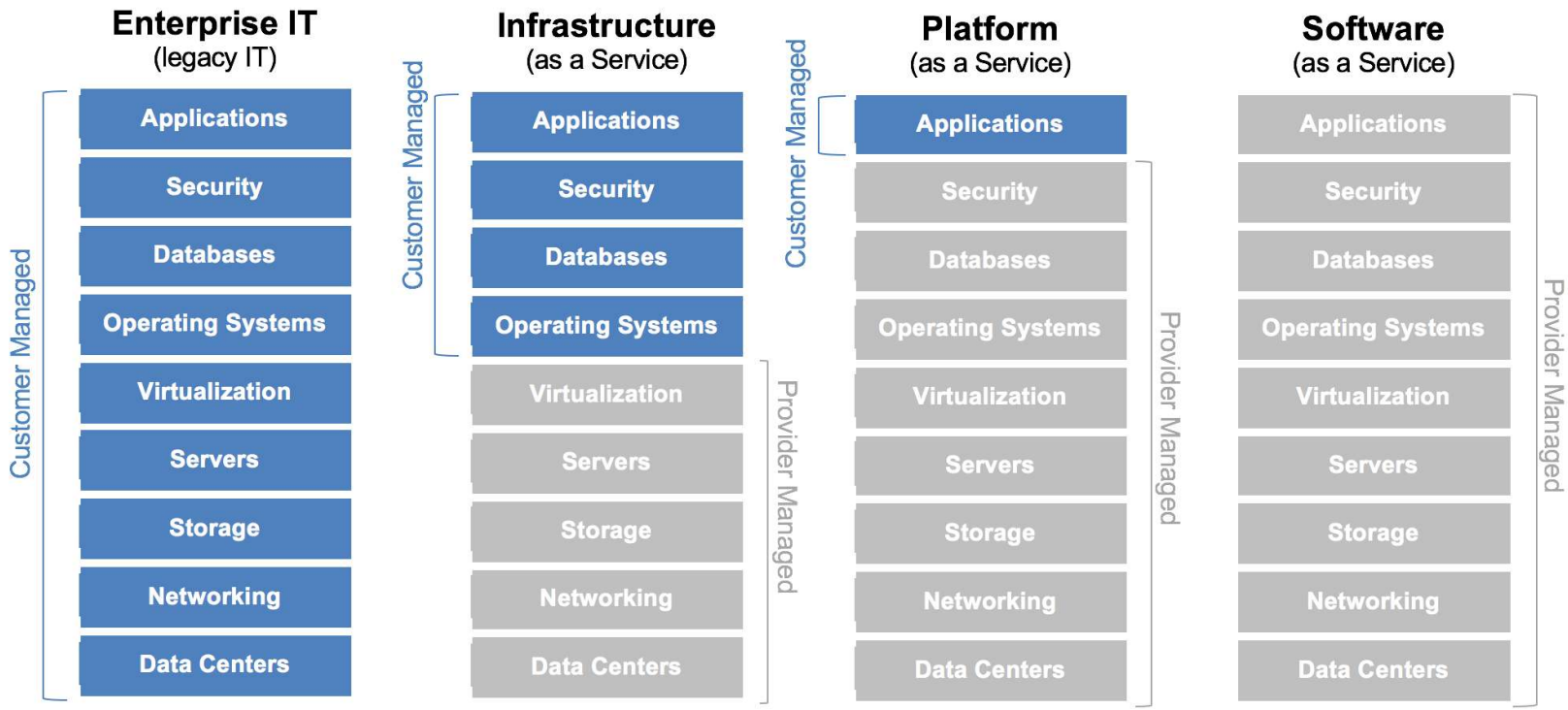
- A technology that allows us to create virtual versions of hardware and software that operate in isolated execution environments.
  - Storage virtualization (aggregation of multiple storage devices into what appears to be a single storage device)
  - Server virtualization (partitioning of a single physical server into smaller virtual servers and hosting multiple clients)
  - Network virtualization (using network resources through a logical segmentation of a single physical network)
  - Operation systems virtualization (a type of server virtualization at the kernel layer)
- *Partitioning the shared pool of resources and dedicating slices of this to different services or different tenants.*

Virtualization Software

- Virtualization software or virtual machine managers
  - Type 1 hypervisor or bare metal hypervisors
    - Run directly on top of the actual hardware resources
    - E.g., VMware Fusion, Parallels and VMware Workstation
  - Type 2 hypervisor
    - Client installed and installed on top of an existing operating system
    - E.g., VMware ESX/ ESXi, Xen, Oracle
- Main difference:



Cloud Computing Service Models



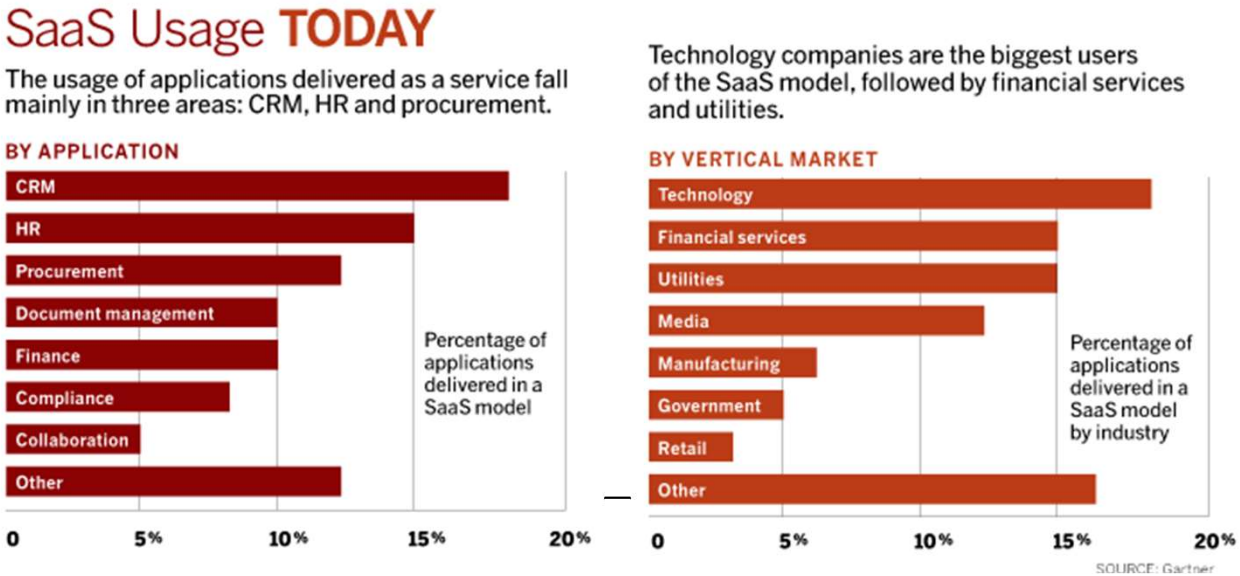
SaaS

- Traditional software model:
  - Purchase a software license and install it on the client's host machine
    - One-time cost for the client
    - Future upgrades to newer software versions would either be free or come at some additional cost

- **SaaS:**
  - On-demand model
  - "Pay-as-you-go" over contract period (months or years)
    - Some SaaS provide a usage-based billing by the minutes (e.g., Next Phase Virtual Call Center)

SaaS

- SaaS applications:
  - Customer relationship management (CRM) – Salesforce
  - Accounting and financial management, Sales, Expense management – ADP payroll and HR solutions, WorkdayHuman Capital Management and SAP SuccessFactors HR
  - Virtual collaboration – Google Apps, Citrix's GoToMeeting, Cisco's WebEx



PaaS

- A cloud computing service that facilitates quick software development without investments in hardware and operating system
  - Can simply upload and develop software code on the cloud platform
  - E.g., shared web hosting services that will allow you to create and host your own web page; app hosting services
  - E.g., storage services, notification and payment services
  - Amazon Web Services, Google App Engine and Microsoft Azure, Salesforce, Heroku

PaaS

- PaaS is especially useful in any situation where:
  - Collaborative development effort across multiple software developers
  - Need for automated testing and deployment
  - Development of agile software
- Not suitable when:
  - Application needs hosting portability
  - Proprietary approaches can impact on the development process
  - Application performance requires customization of the underlying hardware and software
- Considerations include assessing vendor's long-term health, lock-in potential, and I/O performance

IaaS

- Traditional model:
  - Company telephone PBS system
    - Physical fixed line telephone system (e.g., from Nortel, Bell systems)
    - Install it within the company's premise
    - Purchase and interconnect all the telephony devices to telephone exchange system → expensive and labor intensive

- IaaS: a way of delivering cloud computing infrastructure (i.e., servers, storage, network and operating systems) as an on-demand service
  - With hosted voice over IP (VoIP) solutions, instead of buying the physical switch box, the client can have VoIP-enabled telephones on every office desk that connect to a central server of the IaaS provider that is hosted on the Internet

Background - Software Ecosystem

- Software ecosystem is defined as “the interaction of a set of actors on top of a common technological platform that results in a number of software solutions or services”. Software ecosystems have three main aspects: organization, business and software.

- Software ecosystems can be put into three categories:
  - Operating system-centric
  - End-user programming
  - Application-centric

Background - Software Ecosystem

- Software ecosystems can be put into three categories:
  - **Operating system-based software ecosystems** are domain-independent and assume that third-party developers build applications that offer value for customers.
  - A specific category under **end-user programming** category explicitly focuses on application developers that have good domain understanding, but no computer science or engineering degree. Such a capability can be offered for instance, by providing a very intuitive configuration and composition environment (in other words modelled in terms of the concepts of the end-user) that the end-user can create the required applications himself/herself.
  - **Application-centric category** is domain-specific and often starts from an application that achieves success in the marketplace without the support of an ecosystem around it.

Background - Service-Oriented Architecture

- SOA is the technical enabler of the open systems which enables reusability of services and standardization efforts to interactions. SOA brings service providers, consumers and brokers together, which is a means of facilitating inner and inter-organizational computing by reusing services and service descriptions while relying on architecture.

- With the use of SOA, adaptability and flexibility in design and runtime have become emerging research issues. The basic goal of SOA is to design a system that provides a set of services to end-users and other services for fulfilling business needs. Services are orchestrated to realize business processes helping business-IT alignment with a well-understood architecture.

- As every application has a different architecture outlined by orchestration and/or choreography, service composition plays an important role to achieve flexibility to respond to rapid changes.

Background - Service Composition

- In many real-world applications, it is not possible for a single service to meet existing functional requirements. This fact exists despite the presence of complex and discrete services. That's why it's important to have a group of simple services that work with each other in a complementary manner towards achieving complex systems. Moreover, a service composition mechanism is required to be embedded in cloud computing.

- Many simple services provided by different service providers and a variety of effective parameters in the cloud pool have made service composition an NP-hard problem. The existing methods for tackling this problem spread into five categories: classic and graph-based algorithms, combinatorial algorithms, machine-based approaches, structures and frameworks. Despite the existence of a variety of methods, only a limited number of these potential solutions can offer near-optimal solutions to specific problems.

Background – Cloud Applications

- A cloud application, or cloud app, is a software program where cloud-based and local components work together. This model relies on remote servers for processing logic that is accessed through a web browser with a continual internet connection.
- Cloud applications use a client-server architecture.
- Assuming a reasonably fast internet connection, a well-written cloud application offers all the interactivity of a desktop application, along with the portability of a web application.

Cloud Application Qualities

- Cloud application qualities refer to the key characteristics or attributes that define how well a cloud-based application performs, scales, and serves its users in a cloud environment. These qualities ensure the application is reliable, scalable, secure, and cost-effective when deployed on cloud infrastructure.
- Consumers usually do not know how to technically explain what they need, and it is overwhelming for software engineers to remember all the cloud qualities and details when gathering the requirements

Key Drivers of Cloud Applications

- To develop and deliver resilient, high-quality cloud applications, software engineers must understand the key motives that attract enterprises to cloud applications. This is important because these reasons are likely to impact many other related decisions when developing cloud applications. We call these reasons drivers.

- Drivers define the differences between cloud and non-cloud-based applications that motivate moving applications and data to the cloud. Drivers are used to investigate and identify the desired cloud application qualities that helped attain these drivers.

- The key drivers of cloud applications can be described as follows:

Key Drivers of Cloud Applications

- Lower Capital Costs
  - Enterprises are moving toward outsourcing non-business core services to focus on their core business goals without having to invest their own resources in non-core business activities. Cloud applications are available on demand on a pay-per-use basis and can be accessed through the Internet ubiquitously.
- Higher Adoption
  - Increasing application's adoption is another key driver that causes organizations (i.e., software manufacturers) to think about delivering applications through the cloud. SaaS adoption rates are continuing to increase among large, medium, and small businesses alike.
  - However, there is no guarantee consumers will find, like, and adopt the service; therefore, service marketing and discoverability play a significant role in increasing the number of adopters.

Key Drivers of Cloud Applications

- Easier Upgrades/Updates
  - Providers can provide frequent updates with zero downtime, which means the updates are smaller, faster, and easier to test.
- Better Scalability
  - Cloud application scalability means not just the ability to operate, but also the ability to operate efficiently over a range of configurations.
  - Thus, it is essential to consider cloud application usage patterns and data size when planning cloud application development. Also, scalability helps enterprises to see the value of space and make decisions and plans accordingly.

Key Drivers of Cloud Applications

- Better Technical Support
  - Since cloud applications are delivered to consumers online, usages can be monitored and controlled. Also, feedback and user experience can be collected and analyzed to predict anomalies and provide better support.
- Higher Availability
  - Concepts such as live migration, scalability, and elasticity that emerged with the cloud made cloud-based applications more accessible to a wide range of users.

Key Drivers of Cloud Applications

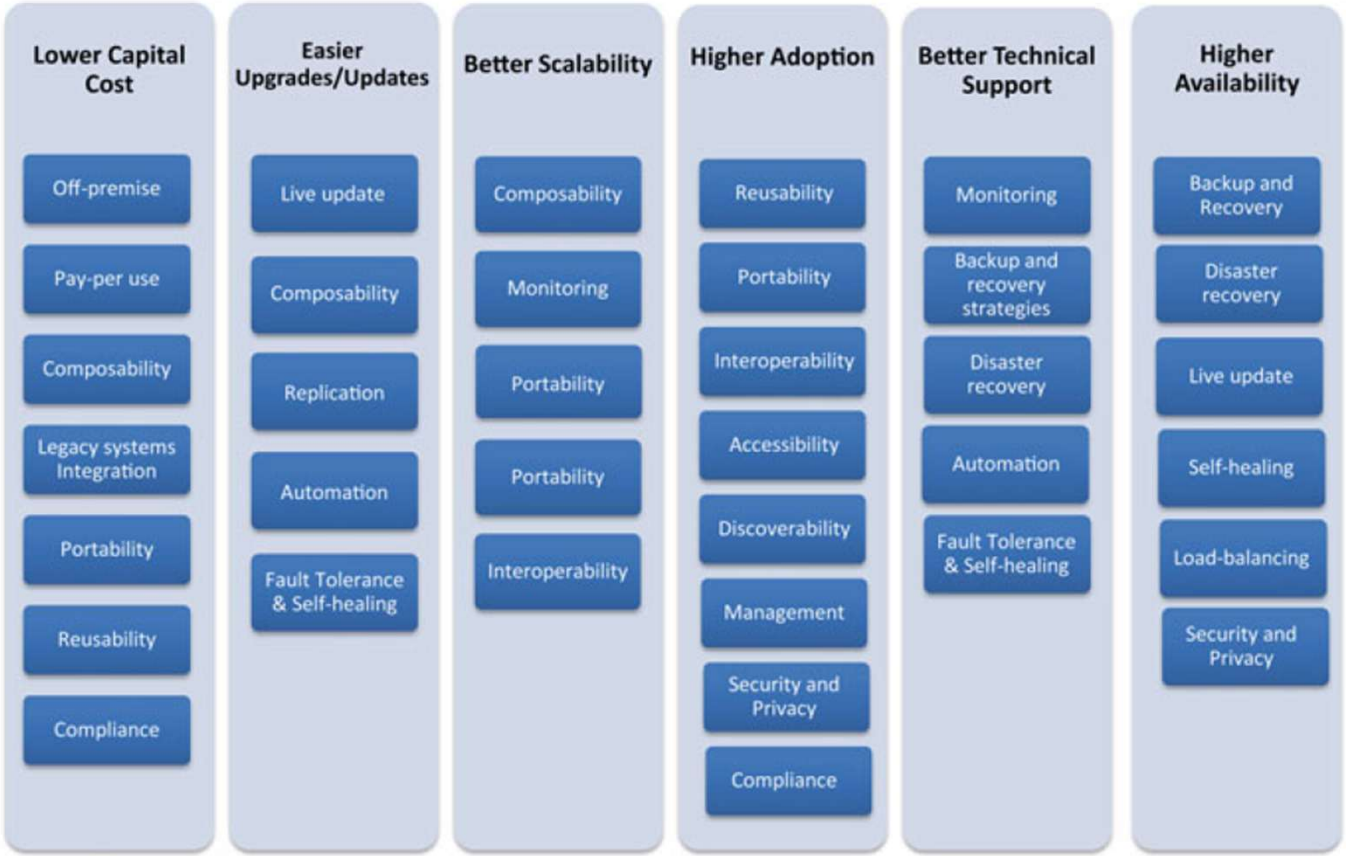


Fig. 2.2 Key drivers of cloud applications mapped to qualities

Cloud Applications Requirements Engineering

- Requirement Engineering (RE) is the **systematic process of defining, documenting, and maintaining the requirements** for a software system.
- The requirement phase in any software development project is carried out as three processes
  - eliciting the requirements of a feasibility study
  - specifying the requirements
  - validating the requirements.

Cloud Applications Requirements Engineering

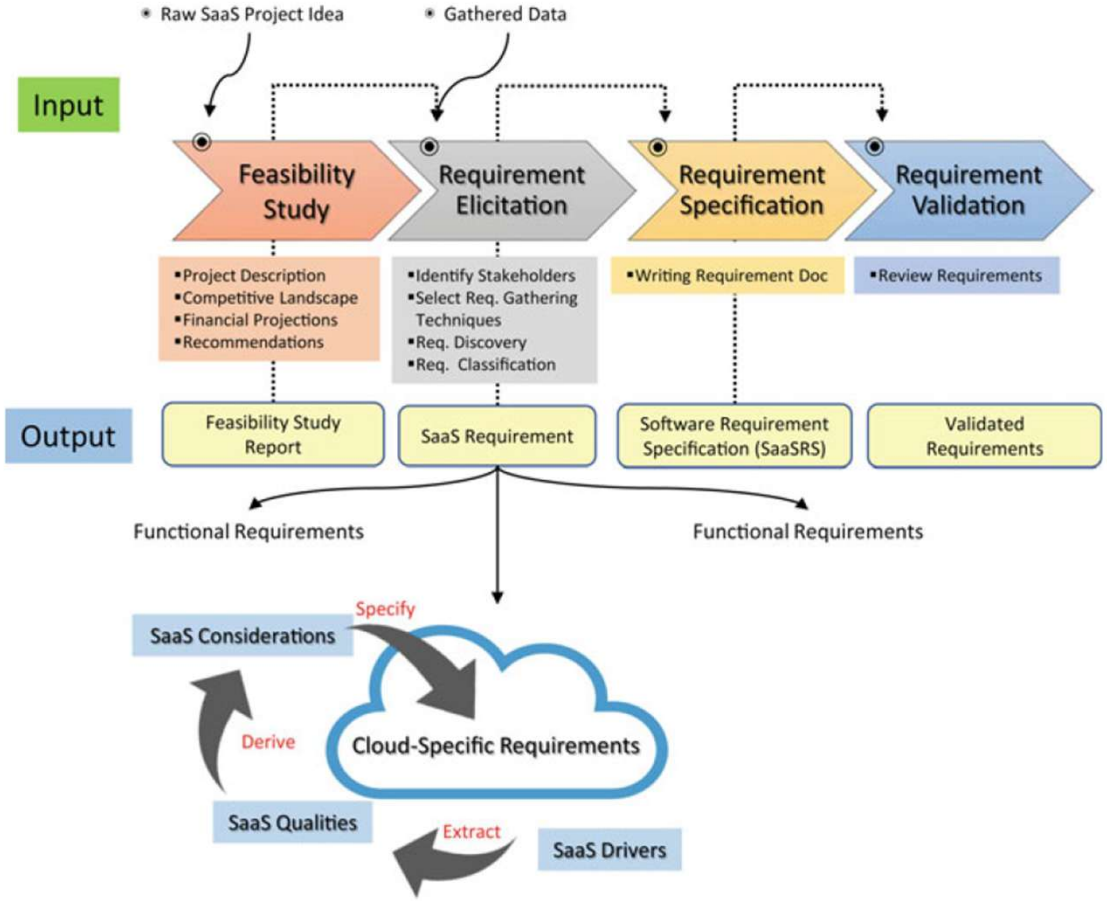


Fig. 2.3 Cloud application requirement engineering process

Cloud Applications Requirements Engineering

- Clouds are used as deployment and delivery mediums, there will inevitably be additional tasks to be planned for as part of the requirement elicitation process.

- These are the tasks associated with cloud-related qualities. As a result, the requirements will be divided into three groups instead of two as follows:
  - Application's Functional Requirements: These describe what the applications should do.
  - Non-Functional Requirements: These specify criteria that can be used to define the operations of a system, rather than specific functions.
  - Cloud Application Requirements: These specify additional functional requirements to be added due to the cloud's use as a medium to deploy and deliver the software service.

Cloud Application Qualities and Requirements

- Off-premise: A measure of the degree to which application location can affect control over the application and its data. This is important because it specifies the hardware and platform on which the software resides, and which is not usually owned by users.
  - What will the IaaS/PaaS configuration and features be?
  - Where will the IaaS be located?
  - What will the response time (latency) be?
  - Who will own, manage, monitor, and control the cloud application's underlying infrastructure and platforms?
  - What is the application deployment model (e.g., private, public hybrid)?

Cloud Application Qualities and Requirements

- Pay-per-use (or on-demand): A measure of the degree to which users can control the cost of using the application. Cloud applications cost a small subscription fee on a per user or usage basis every time the software is being used. It also defines on-demand requests and usage.
  - How many users will access the application?
  - How will the infrastructure scale up/down and charge accordingly?
  - How will cloud application usage be measured?
  - How will underlying IaaS usage be measured?
  - What cloud pricing models will be used (e.g., Pay as You Go, Feature-Based Pricing, Free, Ad-Supported)?

Cloud Application Qualities and Requirements

- Composability: A measure of the degree to which components of the service can be combined and recombined into different systems for different purposes.
  - Will the functionality of the software be composed/decomposed?
  - What will the communication models between subservices be, and how will encryption be carried out?
  - Who owns the service? Can the subservice move and run on another infrastructure owned by a different provider?
  - How will usage patterns be collected for the subservices, and what are the thresholds for each to scale up or down?
  - Who will monitor and control the composed services?
  - How to verify reliability, security, and privacy in the composing components?
  - Will the underlying infrastructure support replication of services and/or their components?
  - What will the message exchange pattern be among cloud application components?

Cloud Application Qualities and Requirements

- Legacy systems integration: A measure of the degree to which legacy applications can be integrated and provided to users as a service. Software encapsulation or containerization enable this quality.
  - Will it be possible to transform current systems into cloud applications rather than building from scratch?
  - Will it be possible to maintain the same legacy system functionality?
  - Will it be possible to migrate old (existing) data and their formats?
  - Will the legacy application and its APIs be able to handle users workload?
  - Will the legacy system be able to address security privacy issues emerging from using the cloud?
  - How will the legacy system be connected to GUI and other components?
  - Will the legacy system be able to run the code concurrently?

Cloud Application Qualities and Requirements

- Portability: A measure of the degree to which application transports and adapts to a new environment to save the cost of redevelopment. The purpose is to provide a service with the ability to run across multiple clouds' infrastructures from different cloud environments.
  - Will the portability of a cloud application and its components be considered during design and development?
  - Are there any portability barriers?
  - Will a portability test plan be considered?
  - Will the cloud application and its components be moved from one cloud provider to another?
  - How will compliance issues in the ported application and data be handled?

Cloud Application Qualities and Requirements

- Monitoring: A measure of the degree to which a cloud application and its subservices behavior are self-monitored in terms of the quality of the service (QoS), performance, security, etc.
  - Will the SaaS environment have monitors to monitor performance, scaling user experience and health?
  - Will the cloud application provide a dashboard to monitor status, report (analytics), and visualize service health?
  - Who will monitor, control, and manage cloud application monitors (e.g., provider, end user, both)?
  - Will the monitoring of the cloud application be automated?
  - Who will be responsible for monitoring service health when service is composed?
  - Will the billing of the cloud application infrastructure also be monitored, (e.g., overcharges, outstanding balance, etc.)?
  - Will the resources (e.g., storage and CPU) of the cloud application be monitored?
  - Will the cloud application monitor and report on bug fixes and upgrades?

Cloud Application Qualities and Requirements

- Backup and recovery strategies: A measure of the degree to which the application and/or its components can be efficiently backed up and recovered.
  - Will the cloud application include a backup plan and facility?
  - Will backup/restoration be automated? How often?
  - Will the cloud application allow restoring the data of one user without affecting another's data (granular backup and restore)?
  - Will redundant copies of backups be stored in different off-site locations?

Cloud Application Qualities and Requirements

- Disaster recovery (DR): A measure of the degree to which the application and its components recover to the latest working state after an unexpected shutdown or hardware failure.
  - Will the cloud application include a disaster recovery plan and facility?
  - Will the disaster recovery process be automated?
  - How quickly does the cloud application recover from failure (time to recover)?
  - Will the cloud application be hosted on multiple, secure, disaster-tolerant data centers?
  - What procedures will be followed when one or multiple infrastructures are down (part of the service but not the whole)?

Cloud Application Qualities and Requirements

- Automation: A measure of the degree to which the application and its components are automated. This includes application provisioning, deployment, and management.
  - Which parts of the cloud application will be automated (e.g., discovery, provision, backup, billing, self-healing, rollback, compensation)?
  - Will the service requests service be automated?
  - Will the service upgrade be automated?
  - Will the service charge be automated?
  - Will the monitoring of service be automated?
  - Will the backup process be automated?

Cloud Application Qualities and Requirements

- Fault Tolerance and Self-healing: A measure of the degree to which a cloud application detects the improper operation of software applications and transactions, and then initiates corrective action without disrupting users.
  - Will the cloud application self-heal?
  - Will service be capable of exception handling?
  - Will the service support transactions and rollback?
  - Who is responsible for service failure caused by a subservice of the application maintained by multiple cloud infrastructures? (i.e., governance)
  - Will the cloud application include multiple instances of the same services/components to improve performance?
  - What measures will be implemented to ensure high availability?
  - What measures will be implemented to ensure fault tolerance?
  - Will the cloud application include load-balancing techniques?

Cloud Application Qualities and Requirements

- Live Update: A measure of the degree to which a cloud application can effectively be updated without requiring the service to go down.
  - Will the cloud application or its components require a live update?
  - Will the cloud application support live updates?
  - How will cloud application updates be scheduled?

Cloud Application Qualities and Requirements

- Reusability: A measure of the degree to which a cloud application can effectively be used in constructing new systems.
  - Will the software or its components be reusable?
  - Who will own the subservice? Can the subservice be reused and run on another infrastructure owned and managed by a different provider?
  - Will the service be reused in another region? How many different compliances should the service conform to?
  - What software licensing will there be (e.g., open-source, GNU, MIT, etc.)?
  - Will the application be integrated with third-party components?

Cloud Application Qualities and Requirements

- Interoperability: A measure of the degree to which the cloud application and/or its components can effectively be integrated, despite differences in language, interface, protocol, and execution platform.
  - How will the subservices conform to multiple compliances, and how general should they be (will they cover multiple countries)?
  - Will all services be accessible through APIs?

Cloud Application Qualities and Requirements

- Accessibility: A measure of the degree to which the cloud application can be available globally to serve customers spanning large enterprises and small business alike.
  - Will the cloud application require a content delivery network service for distant geographical areas (e.g., commercial, self-configuring, or private)?
  - Can the cloud application be configured to allow for choosing server(s) located near clients?

Cloud Application Qualities and Requirements

- Discoverability: A measure of the degree to which interested users can discover a cloud application with minimal or without human intervention on the part of the service provider during the entire service adoption lifecycle.
  - Will the cloud application be self-discovered, configured?
  - Will the cloud application have metadata about its capabilities and behavior in a service registry?
  - Will service description in the registry be abstract and loosely coupled with the underlying infrastructure, service logic, and technology?
  - Will service metadata be updated automatically when service is updated?
  - How will discovery authentication and access control be managed?

Cloud Application Qualities and Requirements

- Service Management: A measure of the degree to which the cloud application can be managed, monitored, controlled, and customized.
  - Will the cloud application be customizable (e.g., user interfaces, theme, user experiences, etc.)?
  - Will the cloud application allow for role changes (profiling)?
  - Will the cloud application include a feature to provide technical support?
  - Will the cloud application include measures for cloud economics (e.g., cost-effectiveness, revenue, and cost of operating the cloud application)?
  - Will the cloud application allow for the authorization of tasks?

Cloud Application Qualities and Requirements

- Security and Privacy: A measure of the degree to which security and privacy of the cloud application and its components and subservience can be addressed.
  - Will the cloud application include device and user authentication and access control?
  - Will the cloud application require malware detection?
  - Do cloud application users require access to application logs?
  - Will data in the cloud application require encryption at the client, or while in transit, at rest, and in process?
  - Will the cloud application be delivered through a secure channel (e.g., https, VPN, etc.)?

Cloud Application Qualities and Requirements

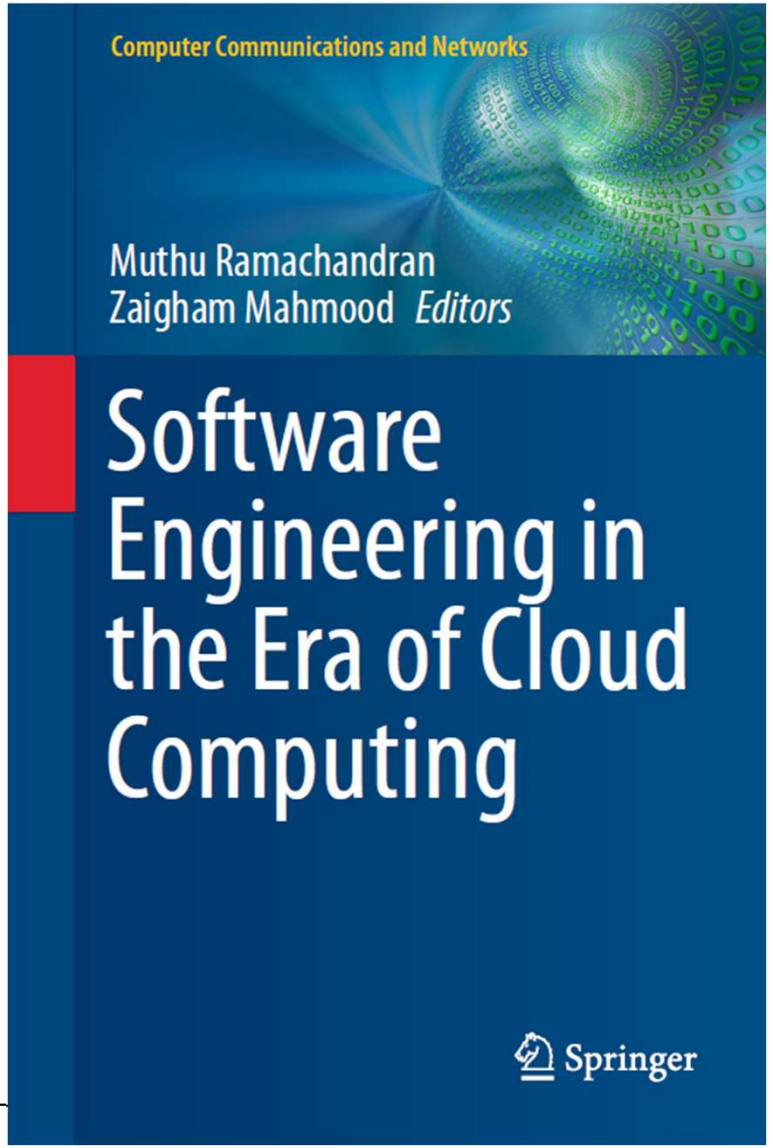
- Compliance: A measure of the degree to which conformance to standards, laws, and regulations is effectively addressed in the cloud application.
  - How will the different compliances affect moving the application and data from one region to another?
  - Can the application conform to multiple compliances?
  - Does the cloud application need to conform to domain-specific compliance (e.g., HIPAA, FERPA, etc.)?
  - Does the cloud application require conformance to global compliance (e.g., ISO, AICPA SOC)?
  - Does the cloud application require conformance to local compliance (e.g., GPDR in Europe, HIPPA in the USA)?
  - Will the cloud application use standard technologies/protocols?
  - Will the cloud application be audited to verify its conformance to SLA?

Conclusion

- Advances in cloud computing have enabled software developers to migrate applications to the cloud environment.
- Various qualities of a cloud-based application may be chosen based on customer requirements without departing from the spirit and scope of the requirement engineering process.
- Additional functional requirements must be added due to the cloud's use as a medium to deploy and deliver the software service.

Reference

- Ramachandran, Muthu & Mahmood, Zaigham (eds) 2020, **Software Engineering in the Era of Cloud Computing** 1st ed. 2020., Springer International Publishing, Cham.



Software Engineering in Cloud Computing II

UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Subject Logistics

(1) Advanced Database Overview
(2) Spatial Data and Computing
(3) Graph Data and Computing
(4) Text Data and Processing

Multi-Modal Database

(1) AI4SE: artificial intelligence for software engineering
(2) Software Engineering in Cloud Computing I
(3) Software Engineering in Cloud Computing II

Software

(1) IoT and Edge Computing
(2) Digital Twin Technology
(3) Blockchains and Smart Contracts
(4) Emerging Cyber-Attacks

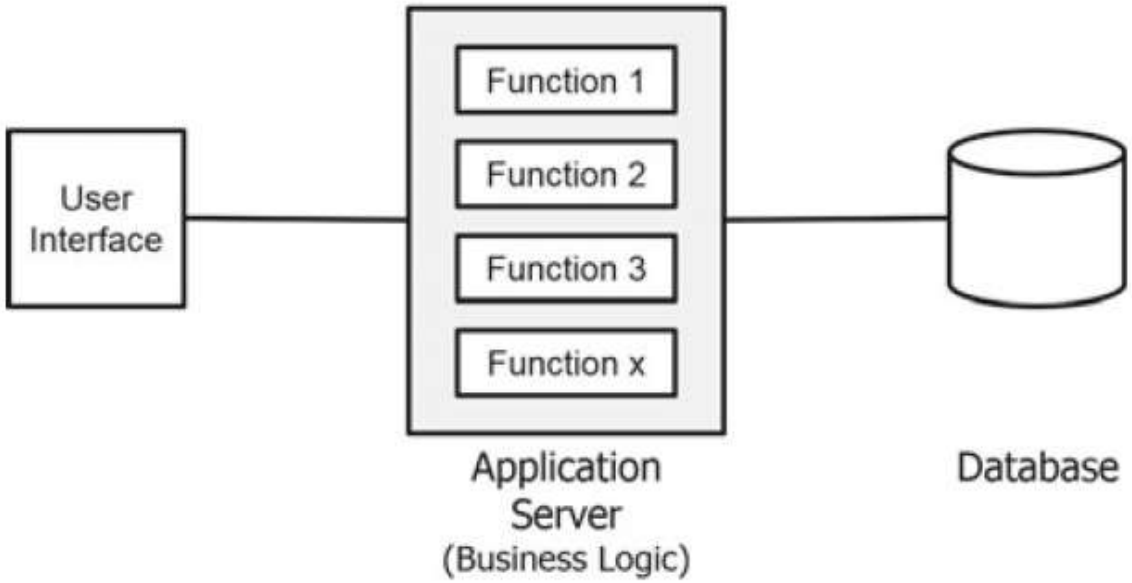
Networking

Background - Monolithic Applications

- A monolithic application, or “Monolith”, describes a legacy style of application architecture which does not consider modularity as a design principle.
- From mainframe to client-server, and to three-tier Web application architecture.
- A common characteristic of all forms of monolithic application is the presence of three distinct architecture layers: the user interface layer, the business logic layer and the database layer.
- All of the business logic is developed and deployed together onto the middle tier, typically hosted on an application server.

Background - Monolithic Applications

Fig. 4.1 Monolithic application (or "monolith")



Background - Monolithic Applications Challenges

- Technology Stack
  - Developers are locked into the original technology stack, and as such are not free to develop new functions using modern application frameworks or languages suitable for cloud deployment.
  - The functions implemented in a monolith, all developed using the same programming language, must interact with each other using native method calls and are therefore tightly coupled.

Background - Monolithic Applications Challenges

- Scalability
  - Functions within a monolith collectively share the resources (CPU and memory) available on the host system. Therefore, scalability within a single monolith is limited.
  - The best and most widely used option would be to scale the monolith horizontally (see the next slide), which adds cost and complexity to a monolith, making it impractical for cloud deployment

Background - Monolithic Applications Challenges

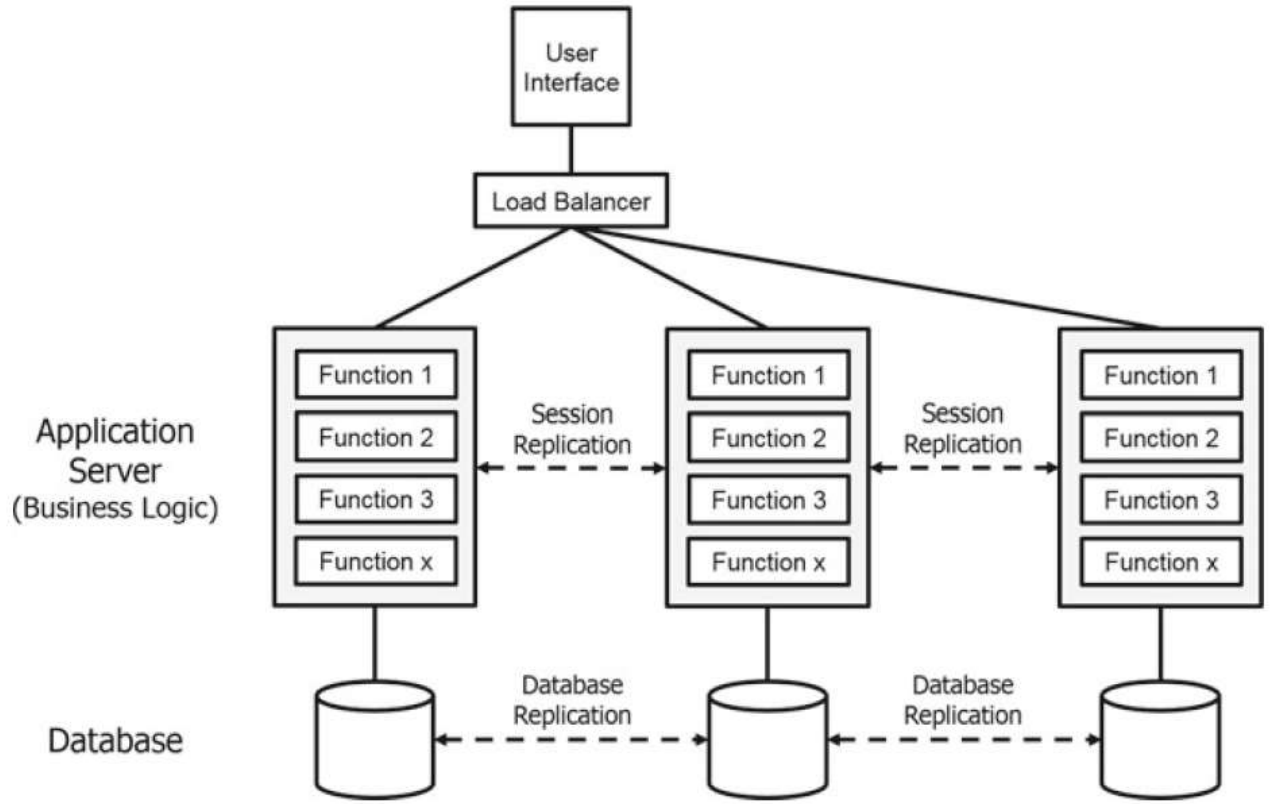


Fig. 4.2 Horizontal scaling of a monolith

Background - Monolithic Applications Challenges

- Change Management
  - Functions which interact using native method calls are tightly coupled and interdependent and therefore are susceptible to change.
  - Any test failures cause the entire application to revert back to the development team for bug fixing.
  - The careful and rigorous testing practices implemented for monoliths can inhibit the time-to-market of new customer experience-driven innovations.

Background - Monolithic Applications Challenges

- Deployment
  - Individual functions within a monolith cannot be individually deployed.
  - Individual functions within a monolith cannot be individually restarted after deployment.

Microservices: A Cloud-based Alternative

- Microservices are a variant of the service-oriented architecture (SOA) architectural style that structures an application as a collection of loosely coupled services.
- A microservice encapsulates a function, or a business capability, which owns its own data, and can be independently deployed and independently scaled.
- A microservice can be atomic (fine-grained) or composite (coarse-grained).
- The communication between microservices can be event-based.

Instead of building a single monstrous, monolithic application, the idea is to split your application into set of smaller, interconnected services.

Microservices: A Cloud-based Alternative

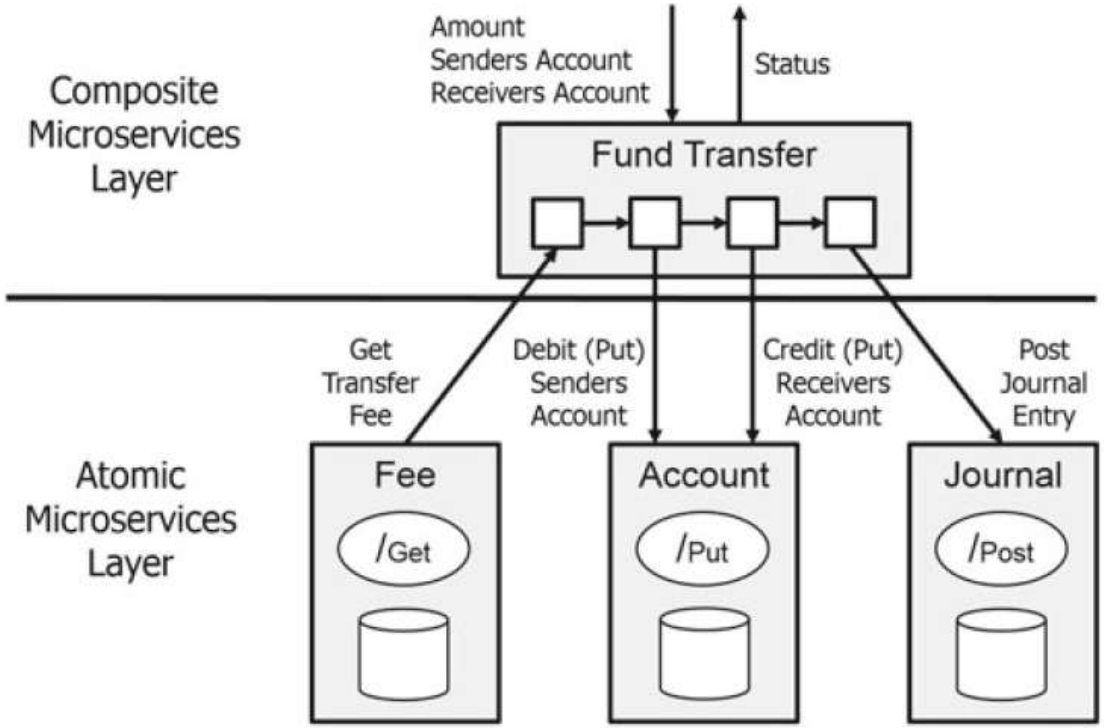


Fig. 4.3 Microservice layers (with fund transfer example)

Microservices Architecture Design Principles

- Do one thing well
- No bigger than a squad
- Grouping like data
- Do not share data stores
- A few tables only
- Independent technology selection
- Independent release cadence
- Limit chatty microservices

Cloud Deployment of Microservices

- Microservices can be deployed independently and can be scaled independently and are small enough in size that automated build, test and deploy scripts can be implemented using agile DevOps methods and tools.
- The small size of the deployment objects also makes containerization practical, using Docker or similar technology.
- At runtime, each instance is often a cloud virtual machine (VM) or a Docker container.

Monolith vs. Microservices Feature Comparison

Feature	Monolith	Microservices
Technology stack	• Locked into original technology stack and framework • All functions developed with one programming language	• Each microservice can be developed using a different technology, fit for purpose or based on developer preference
Scalability	• Functions within a monolith cannot be scaled independently • Horizontal scaling of the entire monolith is necessary	• Each microservice can be scaled independently via containers • Tools are needed for monitoring and managing containers
Change management	• For any small change, the entire monolith needs to be retested • Change/testing processes are complex and time consuming	• Microservices are small and can be tested quickly • Microservices have independent release cadences
Deployment	• Deployment file is large, slow to startup, may incur downtime • Use of agile DevOps methods and tools is not practical	• Microservices can be deployed independently • Use of agile DevOps methods and tools is appropriate

Differences between SOA and Microservices

Aspect	SOA	Microservices
Granularity	Large and can contain multiple business functions	Small, find-grained and focus on a single capability
Communication	Enterprise service bus (ESB)	Lightweight APIs (REST, gRPC, messaging)
Deployment	Tightly coupled, hard to scale independently	Independently deployable
Technology stack	Tends to use enterprise standards (SOAP, XML, WS-* protocols)	Favors **polyglot** (different languages/tech per service), JSON/REST, event-driven systems

*This slide is developed with the help of ChatGPT

Differences between SOA and Microservices

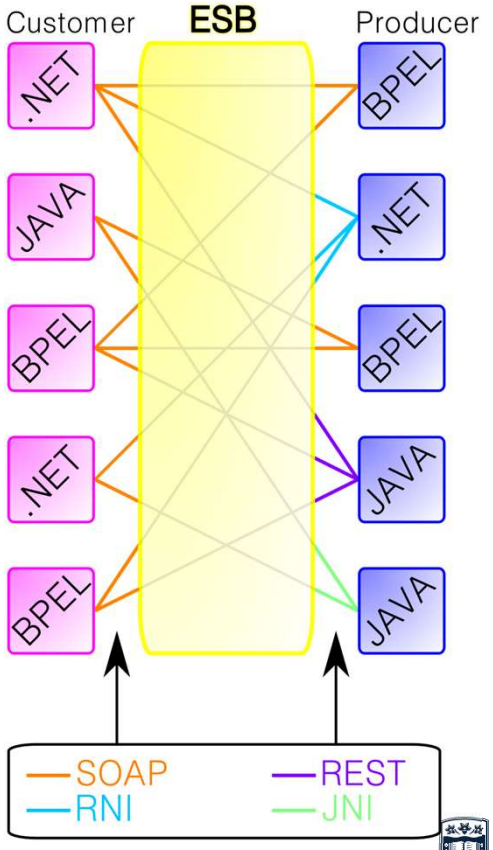
Aspect	SOA	Microservices
Data management	Often shared databases or centralised control	Each service owns its own database
Governance	Centralised governance, standards enforced across services	Decentralised governance — each team owns their service end-to-end
Scalability	Scaling often means scaling the entire service or ESB	**Elastic scaling** of only the microservice that needs it
Use case	Geared towards **integrating enterprise applications** (ERP, CRM, legacy systems)	Geared towards **cloud-native, agile applications** with rapid release cycles.

*This slide is developed with the help of ChatGPT

From Enterprise Service Bus to…

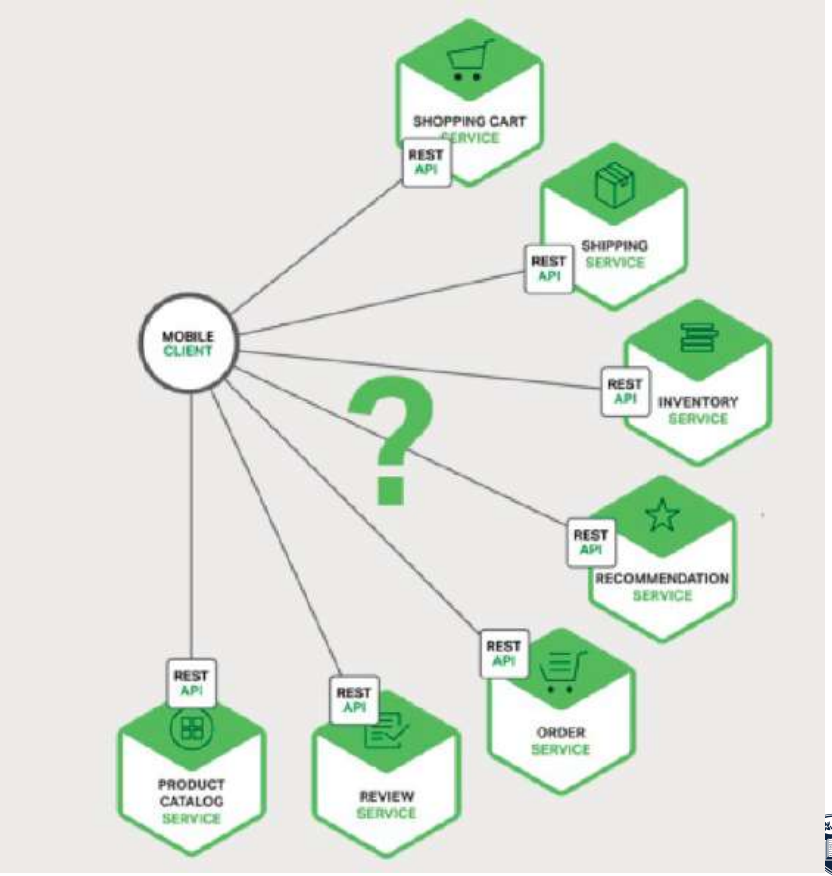
- Centralised hub-and-spoke model
- All services communicate through the ESB
- Creates some dependency on the ESB
- Can handle communication across different technologies (e.g., HTTP, JMS, FTP, SOAP, REST).

- Slower communication speed, especially for those already compatible services
- Single point of failure, can bring down all communications in the Enterprise
- High configuration and maintenance complexity



Accessing Microservices

- Direct client-to-microservice communication is complex and inefficient
- Some microservices. might use protocols that are not web-friendly.
- Difficult to refactor the microservices.



Using an API Gateway

- An API Gateway is a server that is the single entry point into the system.
- The API Gateway encapsulates the internal system architecture and provides an API that is tailored to each client.
- It might have other responsibilities such as authentication, monitoring, load balancing, caching, request shaping and management, and static response handling.

Microservices-based Architecture

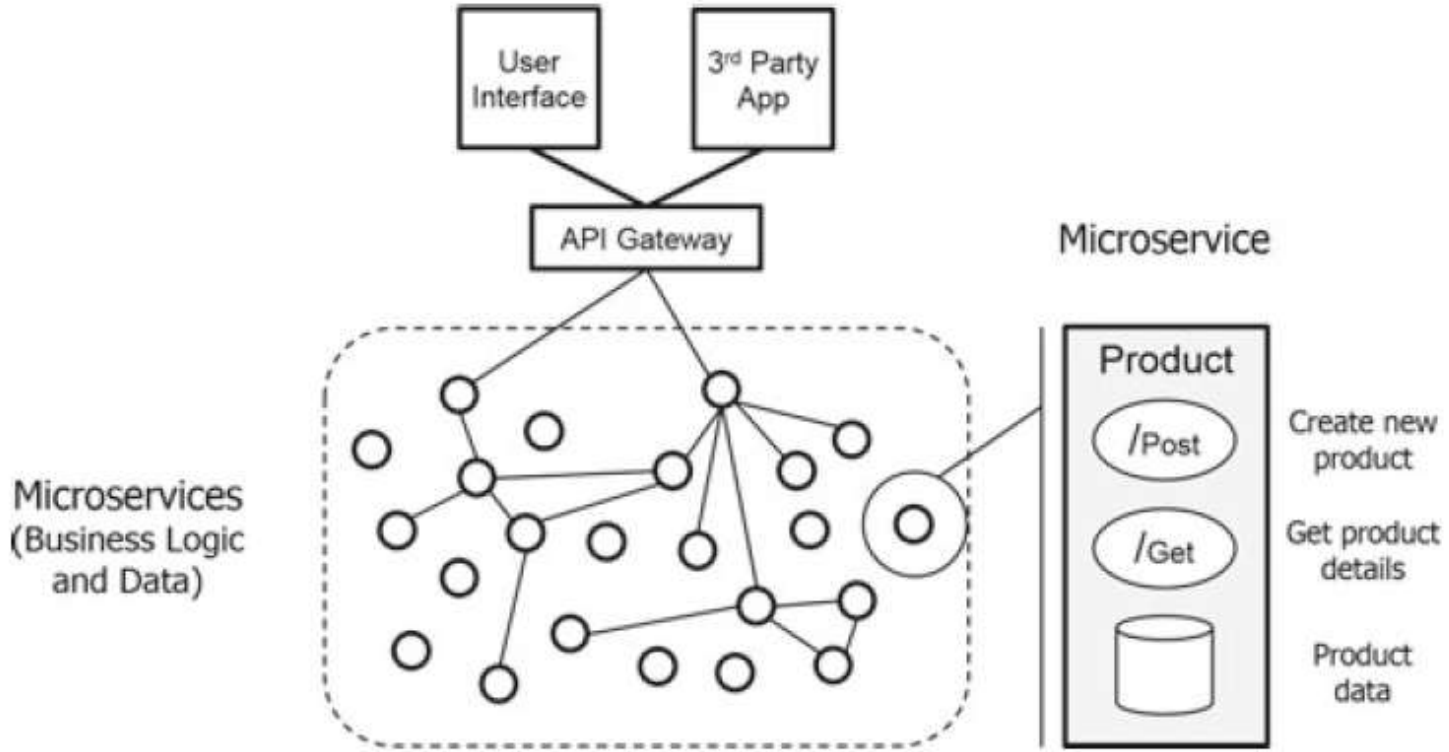
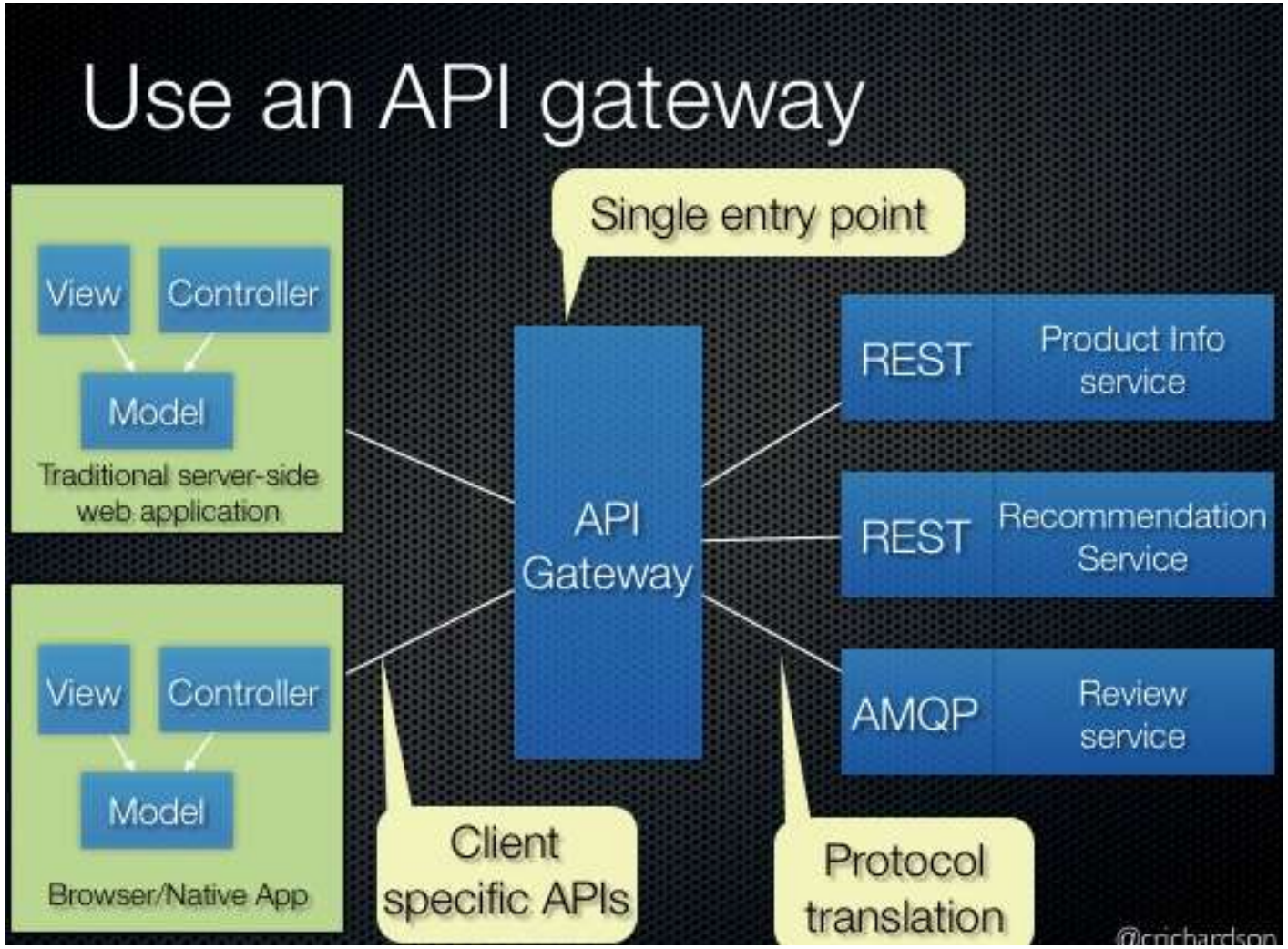
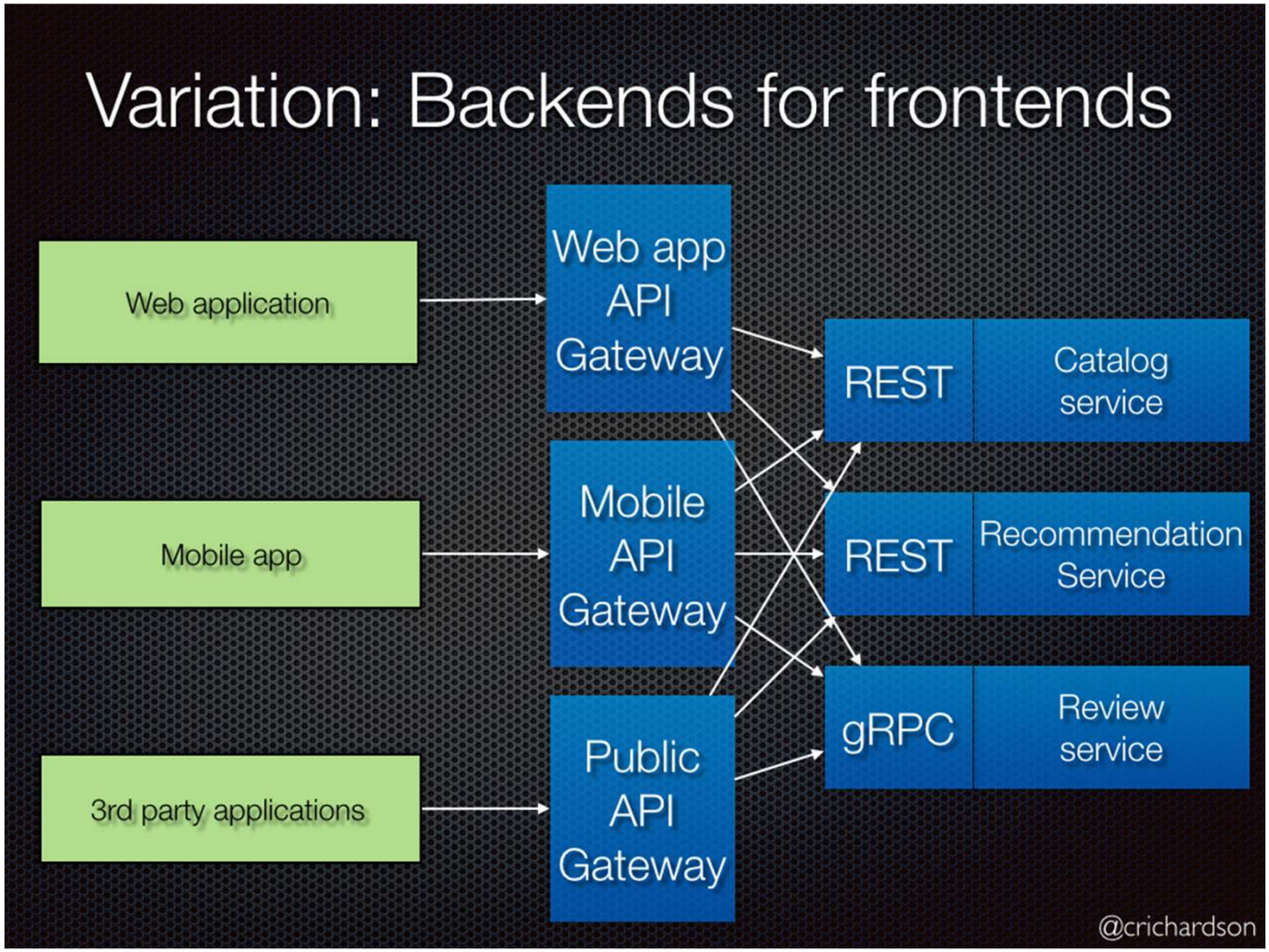


Fig. 4.4 Microservices-based architecture





Benefits of Using API Gateway

- Insulates the clients from how the application is partitioned into microservices
- Insulates the clients from the problem of determining the locations of service instances
- Provides the optimal API for each client
- Reduces the number of requests/roundtrips
- Simplifies the client by moving logic for calling multiple services from the client to API gateway
- Translates from a “standard” public web-friendly API protocol to whatever protocols are used internally

The Downsides

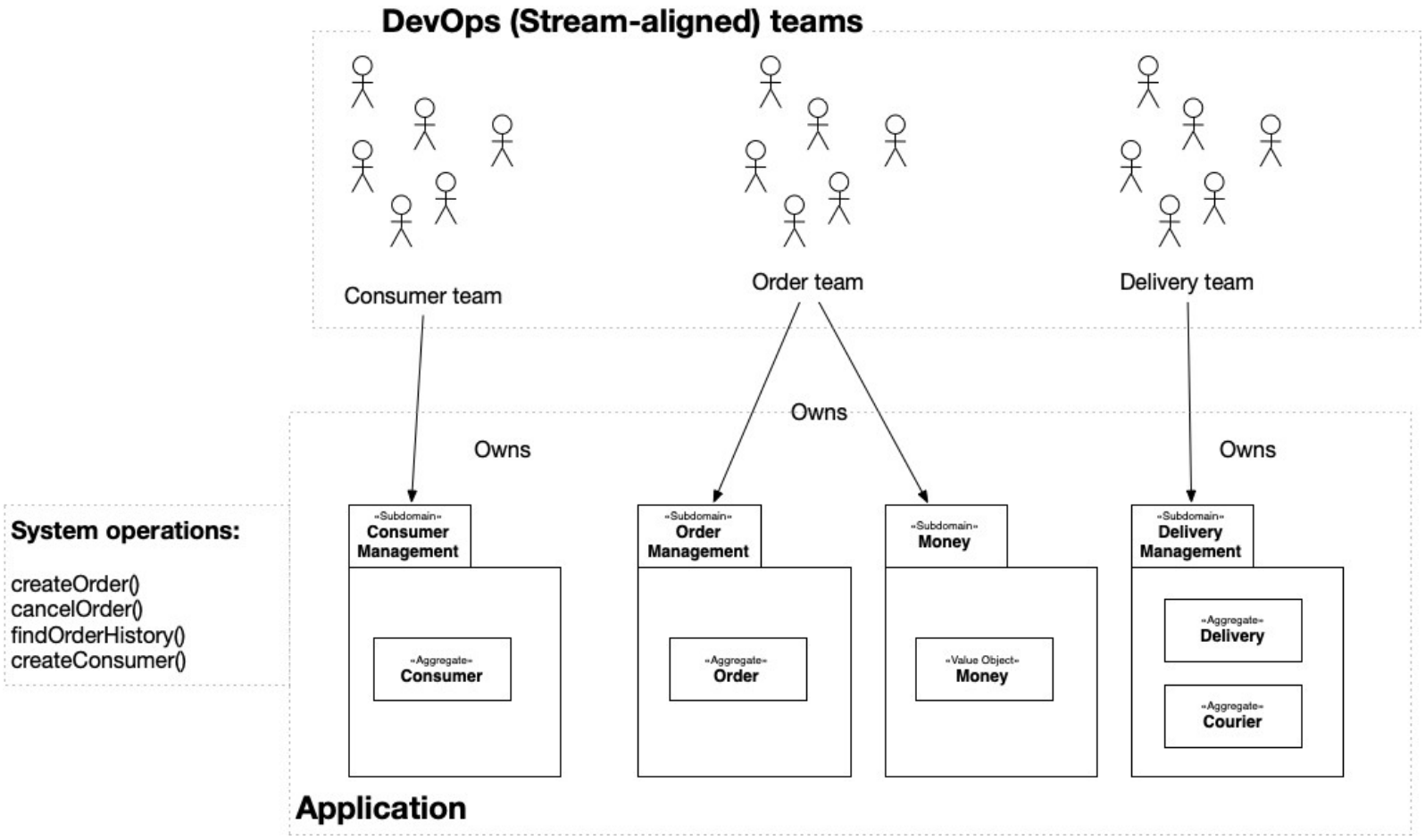
- **Complexity**: The configuration required for routing can become complex when the number of microservices and their API scope increases.
- **Dependency**: Microservices change continuously in terms of API definitions and communication protocols. Every change should be reflected in the API gateway to work seamlessly, bringing yet another dependency to your application stack.
- **Reliability**: API gateways are the entry points to your application stack, meaning they have to be reliable and scalable. If there is only a single API gateway instance and it goes down, it will lead to complete service downtime.

Popular API Gateways

- Kong (open source, widely used in cloud-native)
- NGINX (with API management plugins)
- AWS API Gateway (fully managed in AWS ecosystem)
- Apigee (Google Cloud's API management platform)
- Istio (service mesh) – provides gateway features along with service-to-service security and observability

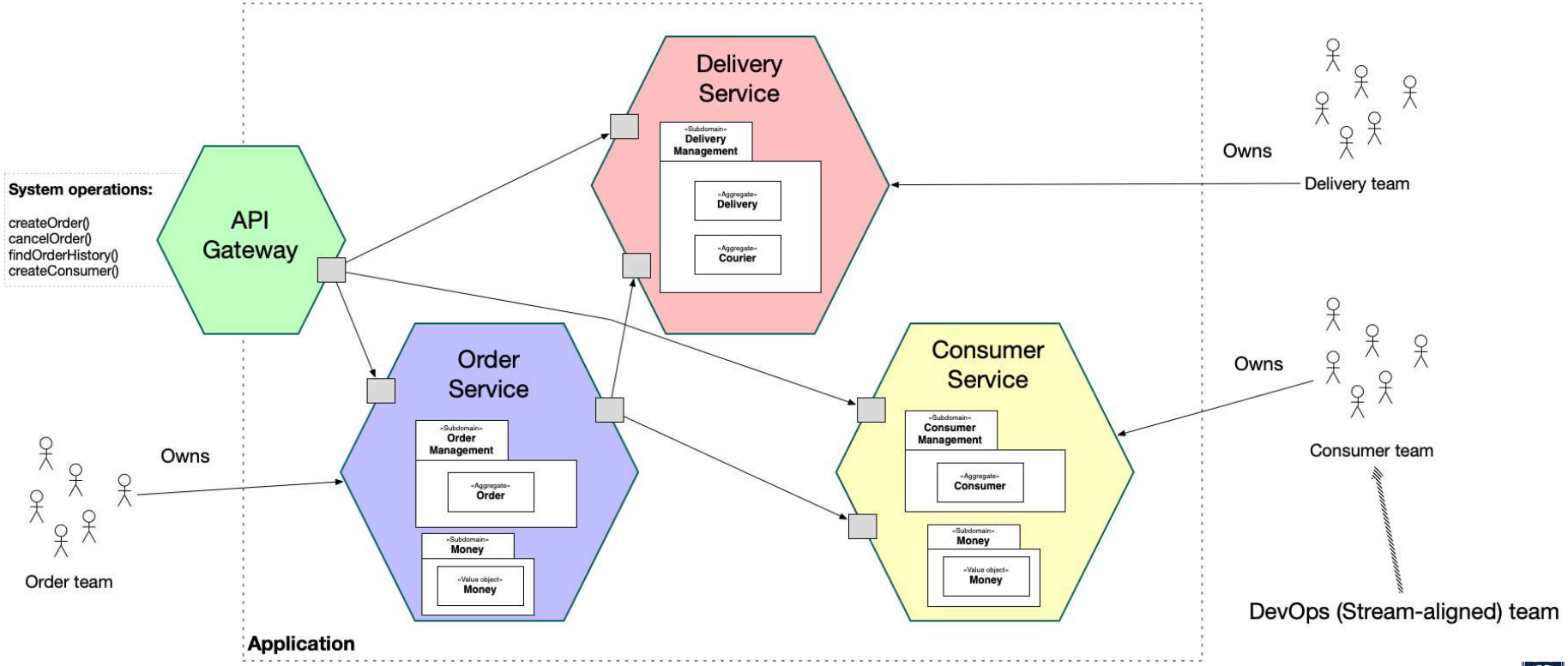
Illustration

- You are developing a business-critical enterprise application. You need to deliver changes rapidly, frequently and reliably
- Consequently, your engineering organization is organized into small, loosely coupled, cross-functional teams, each of which delivers software using DevOps practices
- It practices continuous deployment.
- A team is responsible for one or more subdomains. A subdomain is an implementable model of a slice of business functionality, a.k.a. business capability.

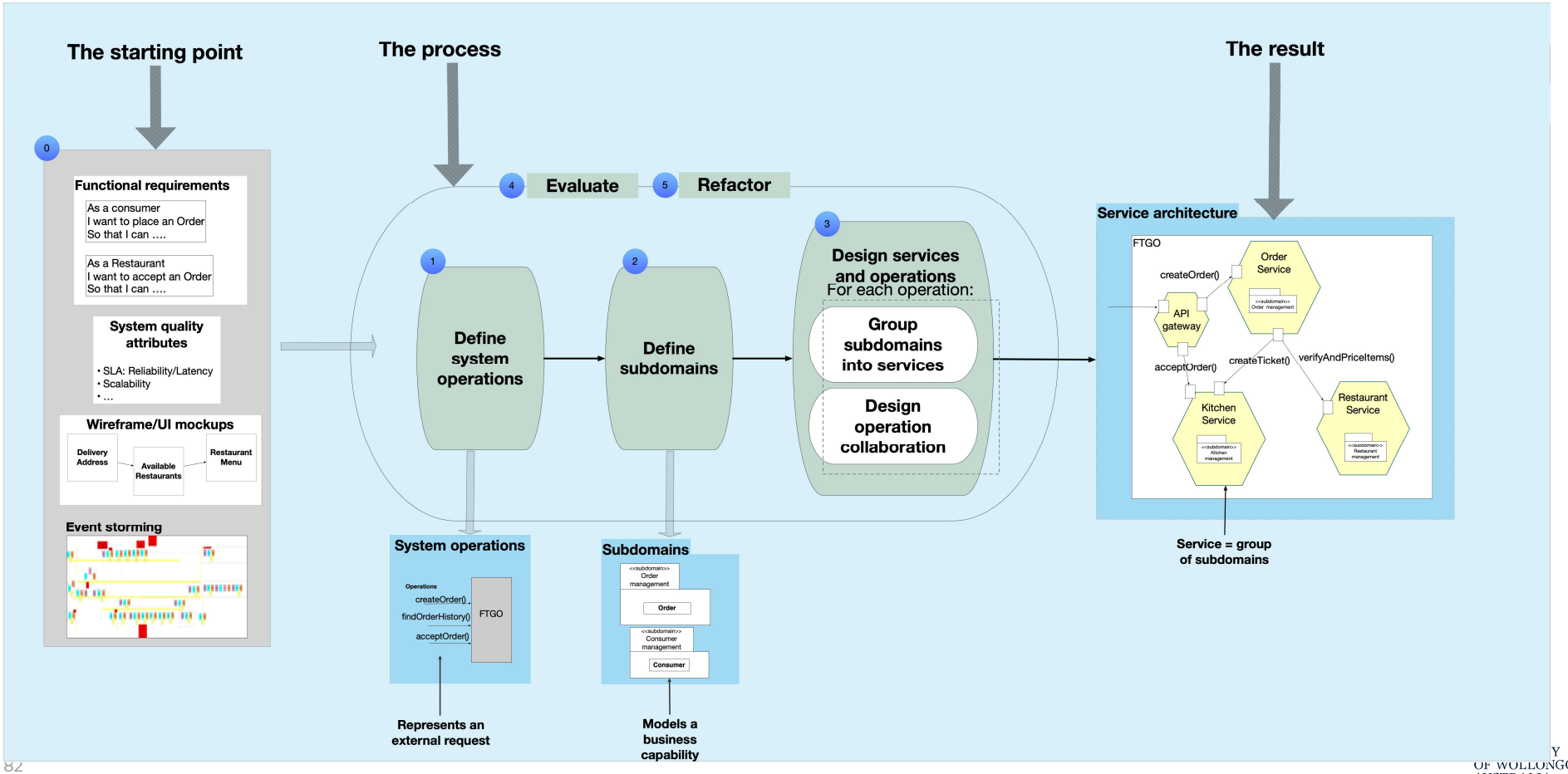


How to organize the subdomains into one or more deployable/executable components?

Solution

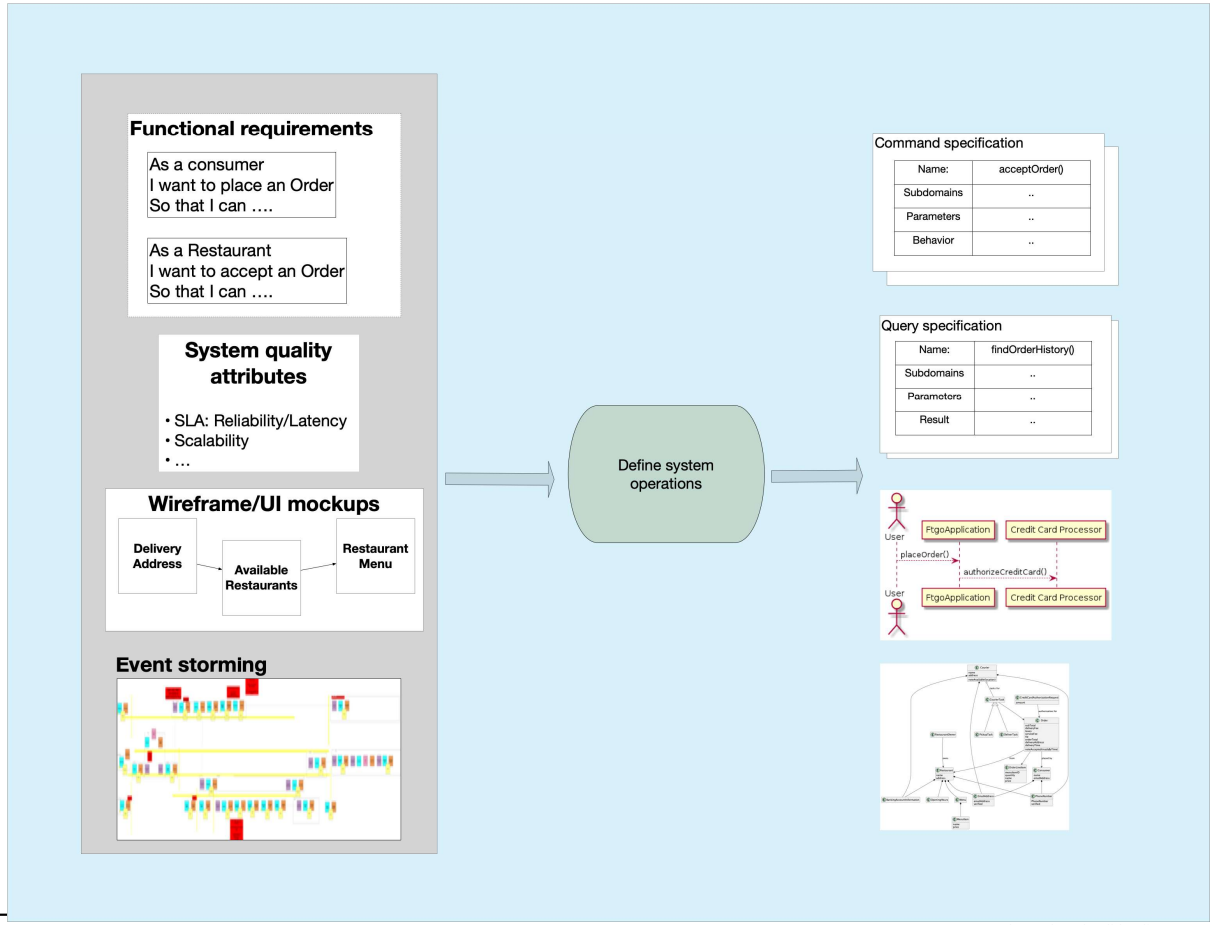


Assemblage: a Microservice Architecture Definition Process



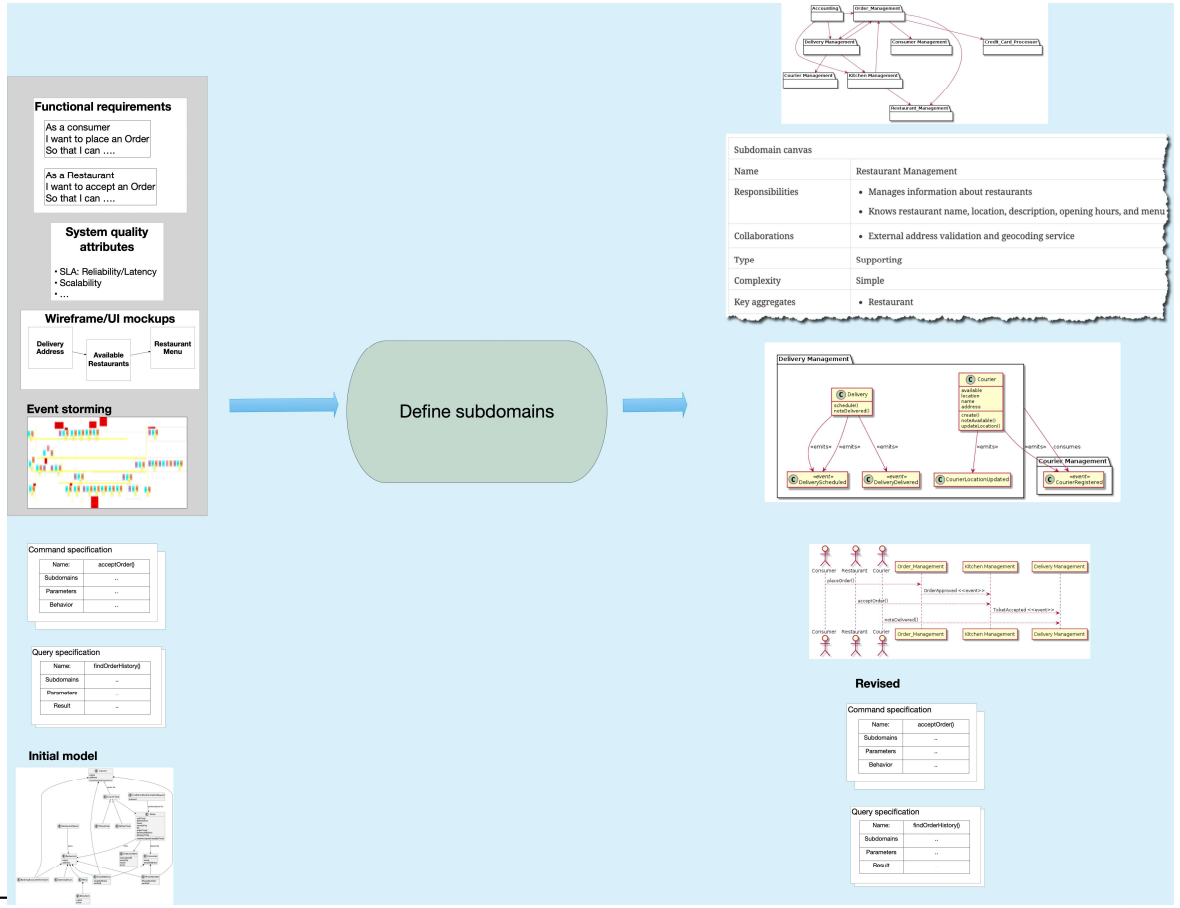
Discovering System Operations

- Distil the requirements into a set of system operations.
  - System operations model the application's black box behavior
  - System operations are invoked by external actors
  - System operations are technology independent
  - System operations drive the architecture definition process



Defining Subdomains

- A subdomain is a team-sized chunk of business functionality, a.k.a. business capability.
- A microservice is a collection of subdomains.

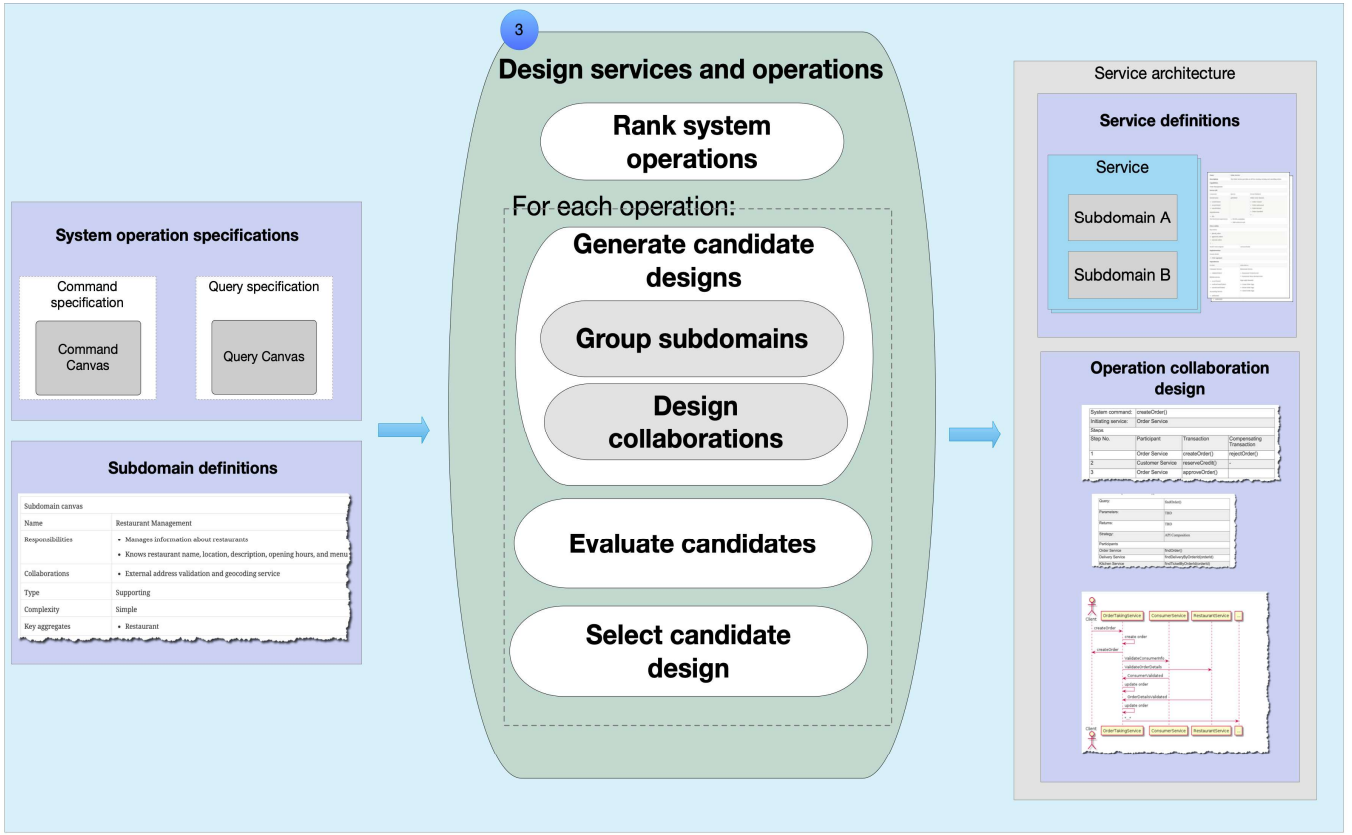


Defining Subdomains

- Talking with domain experts to understand the business, and its structure
- Conducting event storming and using pivotal events and swimlanes to identify candidate subdomains
- Mapping business capabilities identified by business architects to subdomains
- A few things to remember
  - They are a model.
  - They should primarily be aligned to business concepts
  - They should be (Team topologies) team-sized
  - They should be loosely coupled and highly cohesive
  - Each of the entities/aggregates identified in step 1 should be part of a subdomain

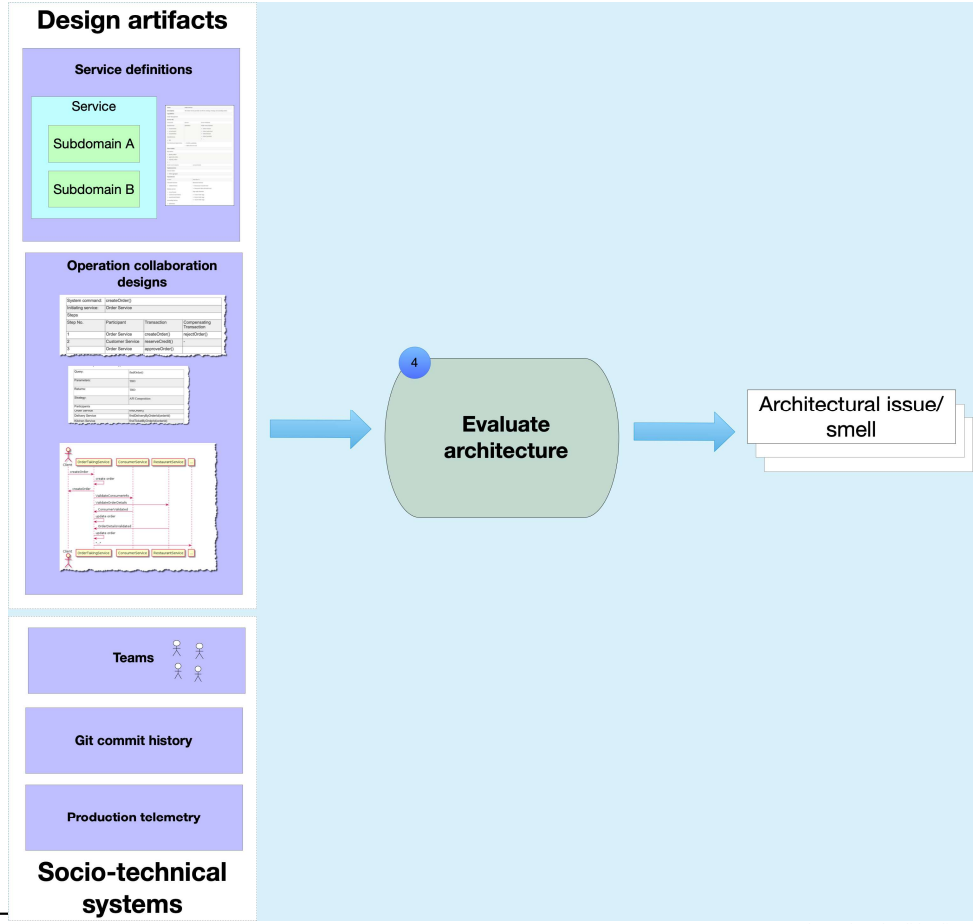
Designing Services and Their Collaborations

- Group the subdomains to form services and design distributed system operations using the service collaboration



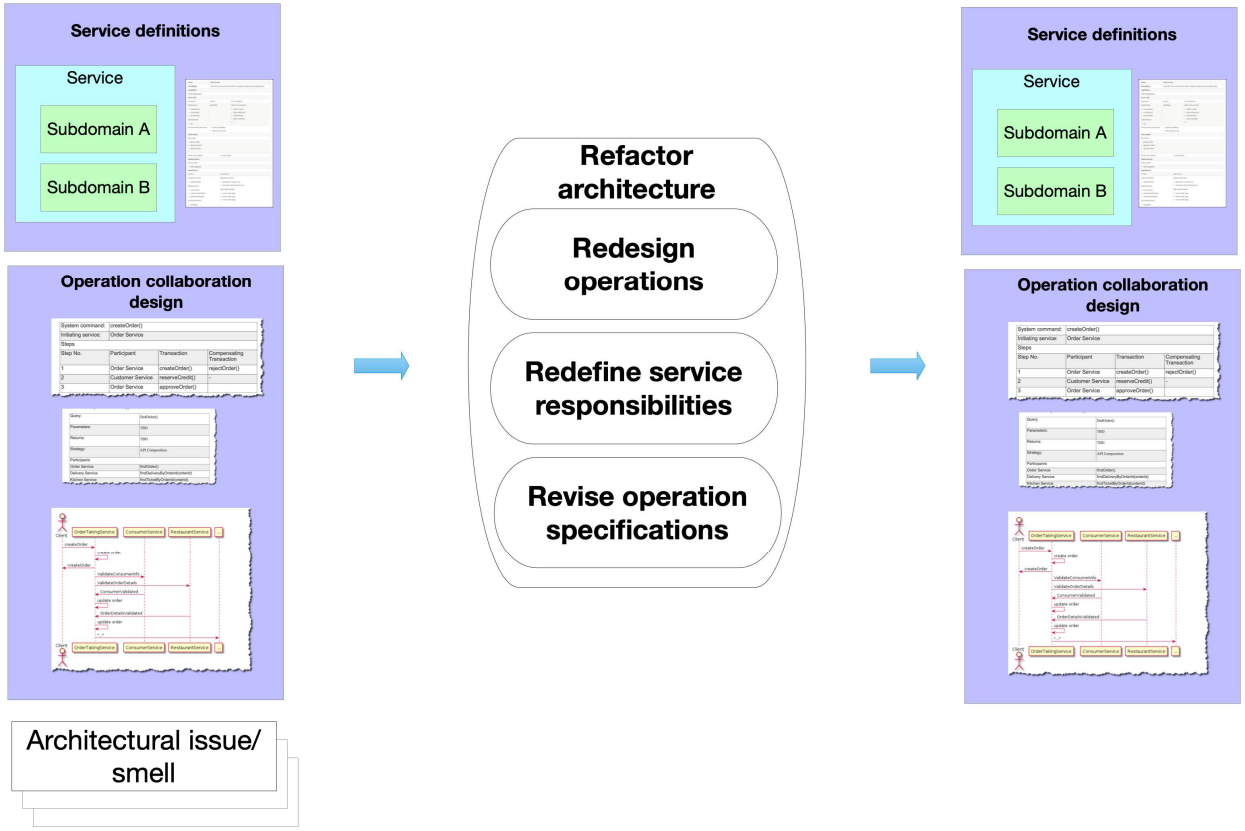
Evaluating the Microservice Architecture

- Evaluate the architecture to identify architectural issues.
  - Teams that lack autonomy because too many teams work together on the same service
  - Services that repeatedly change in lockstep due to design time coupling
  - Operation that have low availability and high latency because they span too many service and require too many network round trips



Refactoring the Microservice Architecture

- Refactor the architecture to eliminate the architecture issues
  - System operations - e.g. change collaboration patterns
  - Services - eg. move subdomains between services
  - Subdomains - e.g. split subdomains
  - System operation specifications - e.g. to reduce runtime coupling

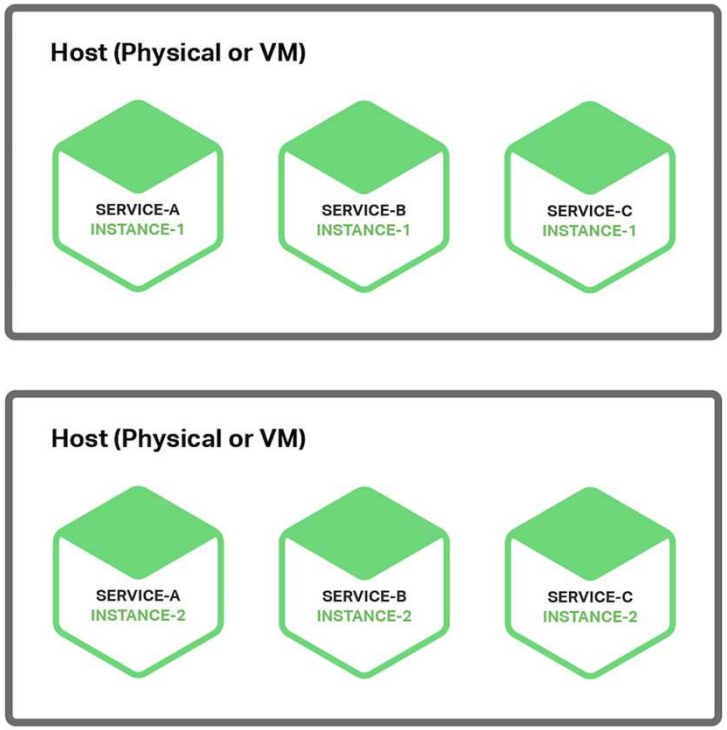


Microservice Deployment

- Deploying a monolithic application means running multiple, identical copies of a single, usually large application.
- A microservices application consists of tens or even hundreds of services.
- Despite this complexity, deploying services must be fast, reliable and cost-effective.

Multiple Service Instances per Host

- Provision one or more physical or virtual hosts and run multiple service instances on each one.
- Each service instance runs at a well-known port on one or more hosts.

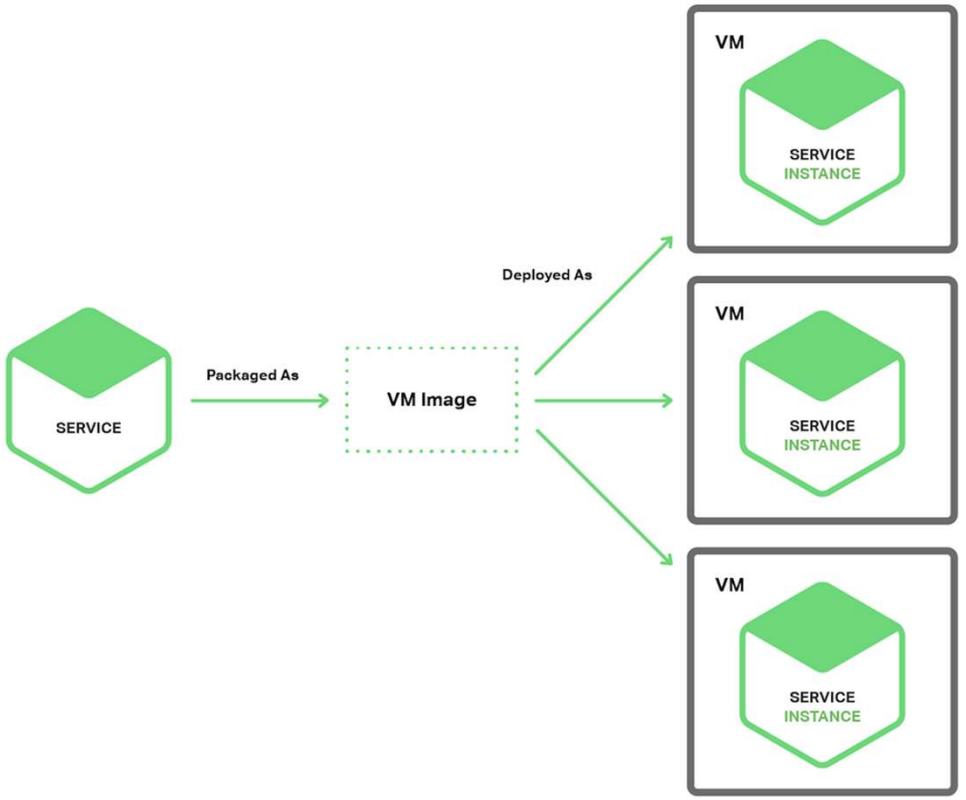


Multiple Service Instances per Host

- Its resource usage is relatively efficient.
- Deploying a service instance is relatively fast.
- Because of the lack of overhead, starting a service is usually very fast.
- There is little or no isolation of the service instances.
- There is no isolation at all if multiple service instances run in the same process.
- The operations team that deploys a service has to know the specific details of how to do it.

Service Instance per Virtual Machine

- Package each service as a virtual machine (VM) image.
- Each service instance is a VM (for example, an EC2 instance) that is launched using that VM image.

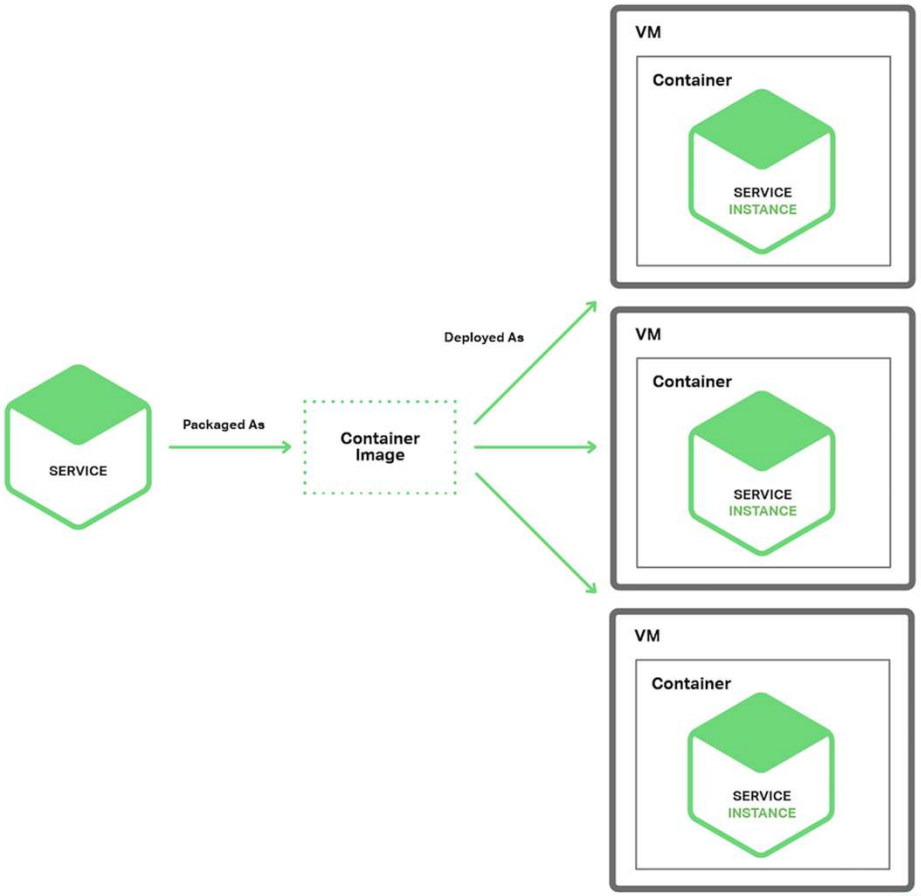


Service Instance per Virtual Machine

- Each service instance runs in complete isolation. It has a fixed amount of CPU and memory and can't steal resources from other services.
- You can leverage mature cloud infrastructure. Clouds such as AWS provide useful features such as load balancing and autoscaling.
- It encapsulates your service's implementation technology.
- Less efficient resource utilization.
- A public IaaS typically charges for VMs regardless of whether they are busy or idle.
- Deploying a new version of a service is usually slow.

Service Instance per Container

- Each service instance runs in its own container.
- Containers are a virtualization mechanism at the operating system level.
- Package your service as a container image which is a filesystem image consisting of the applications and libraries required to run the service.



Service Instance per Container

- Isolate your service instances from each other.
- Easily monitor the resources consumed by each container.
- Containers encapsulate the technology used to implement your services.
- Container images are typically very fast to build.
- Containers are not as mature as the infrastructure for VMs.
- Containers are not as secure as VMs since the containers share the kernel of the host OS with one another.
- Containers are often deployed on an infrastructure that has per-VM pricing

Serverless Deployment

- AWS Lambda is an example of serverless deployment technology. It supports Java, Node.js, and Python services.
- AWS Lambda automatically runs enough instances of your microservice to handle requests.
- A Lambda function is a stateless service. It typically handles requests by invoking AWS services. For example, a Lambda function that is invoked when an image is uploaded to an S3 bucket could insert an item into a DynamoDB images table and publish a message to a Kinesis stream to trigger image processing. A Lambda function can also invoke third-party web services.

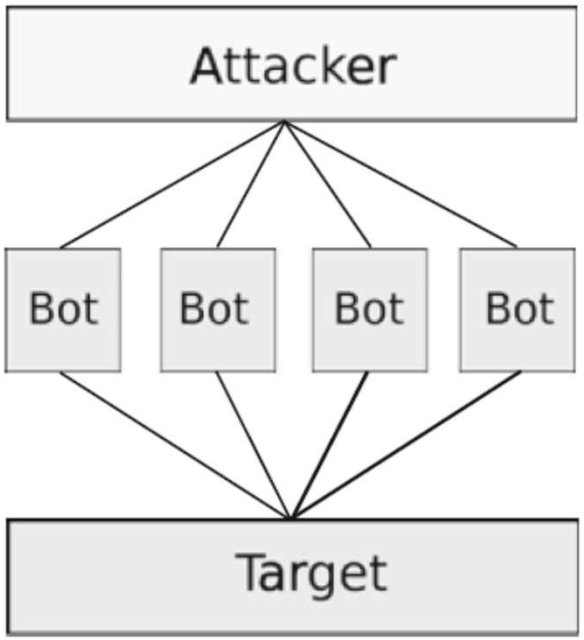
Background

- Cloud systems store a vast amount of data, including sensitive personal information, and as such have become attractive and profitable for malicious actors. Regardless of the severity of the attack, the damage caused by confidentiality exposure from the perspective of the data owner can be devastating.

- It is also worth to note that service models such as software as a service (SaaS) bring new attack surface. In most SaaS deployments, data is stored in localized servers and as such leakage of confidential information is more probable because many of these systems rely on poor user verification processes.

Attacks on Cloud

- Denial of Service Attacks
  - Render the service unavailable
  - Cost the organisation a lot of money for the consumed resources: Fraudulent Resource Consumption (FRC)
- Attacks on Hypervisor
- Resource Freeing Attacks
  - Modify the workload of a victim VM in a way that frees up resources for the attackers VM



Attacks on Cloud

- Side-Channel Attacks
  - Extract secret information from another tenant's VM by monitoring hardware side effects (like timing, cache, or power use) instead of breaking software defenses directly.
- Attacks on Confidentiality
  - Protect data from both public attacks and the service provider
  - Confidentiality is required in the cloud infrastructure as well (encryption keys, VM locations, or operating system information).
  - Including non-technical attacks through social engineering

Security Challenges

- Security challenges that cloud computing users face are clearly seen in software as a service (SaaS) deployment model. Despite cloud benefits, including agility, availability, cost-effectiveness, and elasticity, some of the recent developments have exposed cloud systems to multiple issues.
  - The architecture and design of cloud services have exposed many organizations to many attacks because of the vulnerabilities within their systems.

- The method of deploying some of the cloud service software as well as the technique used to manipulate the data stored within the cloud. From a deployment perspective, the cloud environment combines comprehensive and interdisciplinary deployment strategies and platforms.
  - However, each of these platforms and strategies brings their weaknesses, and most of them do not focus on a conventional design as well as a typical architecture. The lack of uniformity exposes these systems to different attacks and hence the importance of considering software engineering approaches in handling cloud security issues.

Security Challenges

- Security issues in public computing clouds are attributed mostly to external threat although insider threats are also increasingly becoming menacing in cases where mitigation measures and restrictions do not exist or are poor. The security issues are categorized, and threat actors infiltrate the systems based on the layers of a cloud computing platform and the countermeasures placed against unauthorized access.

- The three layers that are susceptible to attack are the infrastructure with about multiple virtual machines, the platform layer, and the application layer. Of these layers, the software layer that serves as the platform for the core of the cloud service as well as the stack for hosting customer application tends to be a pathway for most of the attacks.

Securing Applications on the Cloud

- Securing applications and data on the cloud have its unique challenges because once both data and applications are moved to the cloud, cloud service providers gain access to all the details increasing the risk of misuse of the stored data or hosted applications.
  - Some of the ideal techniques for securing the cloud investing in file distribution systems and securing multiple storage to ensure confidentiality and availability of the data as well as the sources of the applications.
  - Some of the primary methods for protecting data and applications include a regular update of encryption key pairs alongside enforcement of multi-factor authentication, distributing the application to multiple instances within the cloud environment, segregating the virtual machines, and migrating virtual machines of the public cloud to private virtual cloud environments.
- Most importantly, all important source files and data transferred to the cloud must be encrypted to deter man-in-the-middle interception and misuse.

Security Attacks and Recommended Mitigation

Security attacks	Recommended mitigation
Insider threats	Rigorous access control
Eavesdropping	Encryption, intrusion detection and prevention systems (IDS/IPS)
Denial of service (DoS)	Intrusion detection and prevention systems (IDS/IPS)
Man-in-the-middle attack	Firewall, authentication, and IDS
Unauthorized access	Multi-factor authentication and updated public-key cryptography
Virtual machines attacks	Distributing application to multiple instances within the cloud and integration of private virtual cloud environment

Recommended Best Practices

- Some industry best practices with respect to cloud security are highlighted in the list below.
  - User Access Management: Implementation of access control policies, including role-based access control (RBAC), mandatory access control (MAC), and discretionary access control (DAC).
  - Data Protection: Implementation of the web applications firewall, the on-premise firewall, and hardware-based multi-factor authentication, along with strong encryption through hardware security modules (HSMs).
  - Monitoring and Control: Integration of strong intrusion detection and prevention systems over the cloud to prevent intrusions.
  - Updates: Ensure that the cloud, servers, and applications have been provided with the latest updates to security patches, updating user IDs and passwords on a frequent basis.

General Security Recommendations

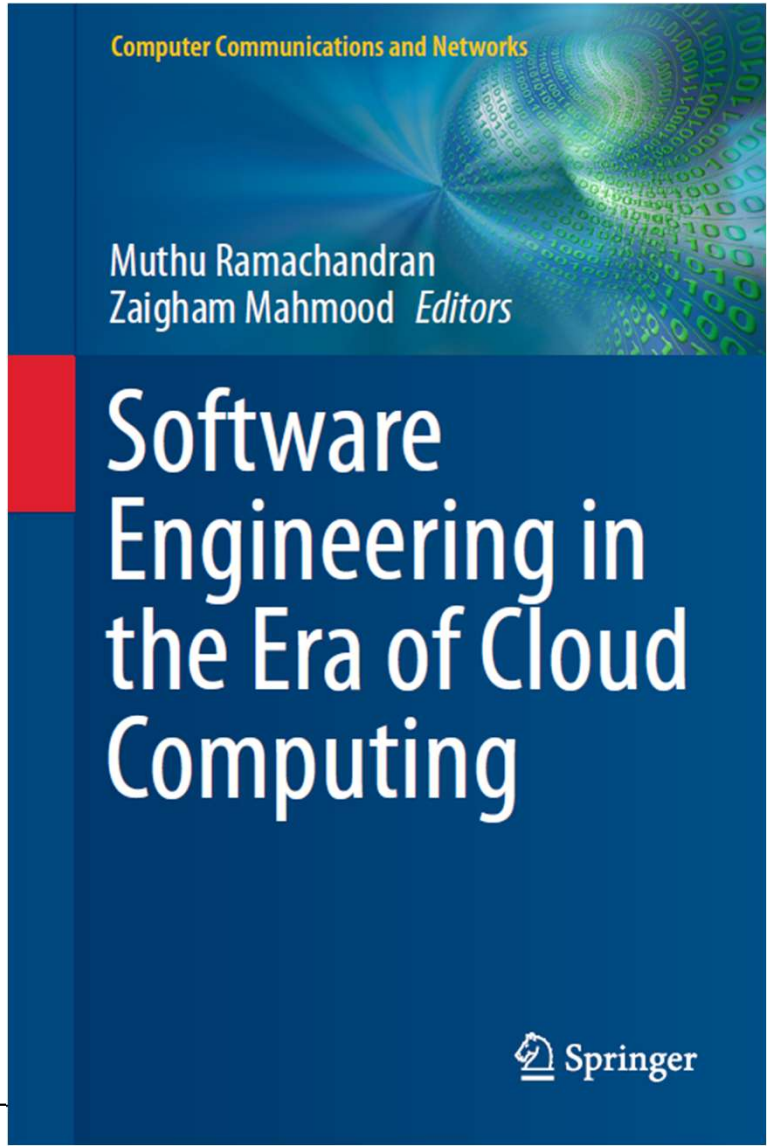
- Install and maintain a firewall configuration
- Do not use vendor-supplied defaults for passwords and other security parameters
- Research into standardized SLAs and liability provisions could lead to greater accountability
- Do not set up your own cloud unless it is necessary
- Ensure that no unnecessary functions or processes are active

General Security Recommendations

- Ensure patch management.
- Give extra attention to a class of attacks called zero-day attacks
- Protect encryption keys from misuse or disclosure
- Promptly revoke access for terminated users

Reference

- Ramachandran, Muthu & Mahmood, Zaigham (eds) 2020, **Software Engineering in the Era of Cloud Computing** 1st ed. 2020., Springer International Publishing, Cham.



Thanks

UNIVERSITY
OF WOLLONGONG
AUSTRALIA